業務工数システム 引き継ぎ教育資料

13 付録 — 出てくるコードとコマンドの一覧

00〜12 と 15 に出てくる 747 個のコードの部品とコマンドを、機能ごとに、意味と初めて出てくる所つきで引く

13 付録: 出てくるコードとコマンドの一覧（機能ごと）

この一覧について

00〜12 と 15 の巻のコードに出てくる、**PowerShell・Git・pip・npm のコマンド、HTML のタグと属性、CSS の項目、TypeScript と React、Python と Django・Django REST framework、pandas・openpyxl（Excel）、テストの道具、書式・記号・設定ファイルなど 747 個**を、**24 の分類**に分けて、1 つずつ説明した一覧です。

- **本文との関係:** どの部品も、本文の中で**初めて出てくるコードのすぐ下**に、「**▶ ここで初めて出てくるコード**」の箱で同じ説明を載せています。この一覧は、それを機能ごとに並べ直した物です。
- **使い方:** 「この `useEffect` は何だっけ？」「`.filter()` は Django と TypeScript で何が違う？」と思ったら、下の「分類の一覧」から分類を選んで探してください。Web 版（`kosu\html`）なら、`Ctrl` ＋ `F` で、コードの名前で探すのがいちばん速いです。
- **「初めて出てくる所」の列:** その部品が最初に出てくる巻と章です（`07 5. 実コード全解説：member_views.py 全2…` は、07 の 5 章。長い章の名前は、後ろを「…」で切っています）。そこで、実際にこのアプリでどう使われているかを見られます。
- **文字や記号の書き方も入っています:** f 文字列（`f'{member_name}氏の…'`）・`\n` のような特別な文字・正規表現・日時の書式（`%Y-%m-%d`）・記号（`?.`・`??`・`&&`・`**kwargs`）・HTML と JSX の属性（`for`・`htmlFor`）・`package.json` の設定・コマンドのオプションも、1 つずつ説明しています。
- **やりたいことから探す時は:** 次の 14 付録「逆引き」で、「一覧をしぼり込みたい」「Excel に書き出したい」のような、やりたいことからコードを探せます。本文に出てこない、合わせて覚えておくとよいコードも載せています。
- **同じ綴りでも、言語が違えば別の物**として、それぞれの言語の分類に載せています。例えば `.filter()` は、TypeScript では配列から条件に合う物を集めるメソッド、Django ではデータベースの行を条件でしぼるメソッドです。
- **分類の並び:** 引き継いだ人が触る順（道具 → Git → HTML・CSS → TypeScript・React → Python・Django → Excel・テスト）に並べています。
- **このアプリが自分で作った名前**（`KosuList`・`handleSearch`・`time_index` など、画面や関数の名前）は、本文のその場所で説明しているので、この一覧には入れていません（よく出てくる共通の部品 `TableContainer`・`Pagination`・`Loading` だけは載せています）。

> **ヒント:** 1 つの部品を覚えたら、その分類の表の、近くにある物も読んでみてください。例えば `Django` の表には、行を作る（ `.create()` ）・探す（ `.filter()` ・ `.get()` ）・数える（ `.count()` ）・並べかえる（ `.order_by()` ）が並んでいます。一緒に覚えると、使える場面がぐっと増えます。

分類の一覧

分類	数	分類	数
PowerShell	9	JavaScript・TypeScript	57
コマンド	2	TypeScript	35
WSL・bash	5	React	66
Python の道具	13	Python	143
Django の道具	6	Django	85
Node.js	11	Django REST framework	20
設定ファイル（JSON）	18	pandas・openpyxl	8
psql	4	正規表現	7
SQL	4	日時の書式	7
Git	40	Vitest・Testing Library	28
HTML	40	Python のテスト	9
CSS	96	GitHub Actions	34

PowerShell

コード	意味と使い方	初めて出てくる所
`cd`	「change directory（フォルダを移る）」の略。`cd Documents` で、今いるフォルダの中の `Documents` へ移ります。`cd ..` は 1 つ上のフォルダへ、`cd ~` はホームフォルダ（ `C:\Users\あなたの名前` ）へ戻ります。今いる場所は、行の頭の `PS C:\Users\…>` の部分に表示されます。	00a 4. コマンド — 命令の言葉
`ls`	フォルダの中の一覧を見るコマンド（正式な名前は `Get-ChildItem` ）。`Mode` の列の `d` はフォルダ、`a` はファイルです。	00a 4. コマンド — 命令の言葉

コード	意味と使い方	初めて出てくる所
`#`	`#` から行の終わりまでは「コメント」（人が読むためのメモ）で、PowerShell は無視します。コマンドの横に、何をするかを日本語で書いておけます。コピーしてそのまま貼り付けても、問題ありません。	01 3. 道具をそろえる②：Git
`mkdir`	「make directory（フォルダを作る）」の略。`mkdir "$HOME\Documents\git-projects"` のように、途中のフォルダもまとめて作れます（ `$HOME` はホームフォルダ `C:\Users\あなたの名前` ）。もう同じ名前のフォルダがあると、エラーで知らせます（ `-Force` を付けると、知らせずに進みます）。	01 7. ソースコードを手元に持ってくる
`mkdir -Force`	`mkdir フォルダ -Force` は、フォルダを作ります。`-Force` を付けると、もう有っても赤いエラーを出さずに進みます。	01 7. ソースコードを手元に持ってくる
`"…$変数…"`	`"…"` （ダブルクォート）で囲んだ文字の中の `$名前` は、変数の中身に置きかわります。`"こんにちは、$name さん"` → `こんにちは、太郎 さん` 。そのまま `$` を出したい時は、`'…'` （シングルクォート）で囲みます。	01 7. ソースコードを手元に持ってくる
`>`	`コマンド > out.txt` は、画面に出るはずの文を、`out.txt` というファイルに書き込みます（前の中身は消えます）。後ろに書き足したい時は `>>` を使います。	01 7. ソースコードを手元に持ってくる
`.\`	`.\venv\Scripts\Activate.ps1` の `.\` は、「今いるフォルダの中の」という意味です。PowerShell は、安全のため、今いるフォルダのプログラムやスクリプトでも `.\` を付けないと動かしません。	01 8. バックエンド準備①：仮想環境
`&`	`& "C:\Program Files\PostgreSQL\16\bin\psql.exe" …` の先頭の `&` は、「この文字が指すプログラムを動かす」という印です。場所に空白が入っていて `"` で囲んだ時に必要です（ `&` が無いと、ただの文字として表示されるだけです）。	01 12. バックエンド準備④：データベースを作ってmigr…

コマンド

コード	意味と使い方	初めて出てくる所
`pwd`	今いるフォルダを表示するコマンド（print working directory）。bash（WSL）では `/home/you` のような Linux の形で、PowerShell では `C:\Users\you` のような形で出ます。	00a 4. コマンド — 命令の言葉
`code`	VS Code を動かすコマンド。`code .` で今のフォルダを開き、`code --version` で版を確かめます。	01 7. ソースコードを手元に持ってくる

WSL・bash

コード	意味と使い方	初めて出てくる所
`#`	bash では、`#` から行の終わりまでがコメントです（1 行目の `#!` だけは特別です）。	02 3. このプロジェクトの Git の状態を見てみる
`source`	`source antenv/bin/activate` は、Python の仮想環境（`antenv`）に入るコマンド（Linux・Mac の書き方）。入ると、`python`・`pip` が、その仮想環境の物になります。	15 7. 開発・運用の整理
`zip`	ファイルやフォルダを、1 つの zip ファイルにまとめるコマンド。`zip -r release.zip .` は、今いるフォルダの中を全部、`release.zip` にまとめます。	15 7. 開発・運用の整理
`zip -r`	`zip` の `-r` は、フォルダの中のフォルダまで、全部まとめるという指定です（recursive）。	15 7. 開発・運用の整理
`zip -x`	`zip` の `-x` の後ろに書いた物は、まとめる物から外します。`-x ".git/*" "node_modules/*"` は、Git の記録と部品のフォルダを入れない、という意味です。	15 7. 開発・運用の整理

Python の道具

コード	意味と使い方	初めて出てくる所
`python`	Python を動かすコマンド。`python -c "…"` は、`"…"` の中の Python のコードを 1 回だけ動かします。	01 4. 道具をそろえる③：Python 3.12
`python --version`	入っている Python の版（バージョン）を表示します。`Python 3.12.x` のように出ます。	01 4. 道具をそろえる③：Python 3.12
`pip`	Python の部品（ライブラリ）を入れたり消したりする道具。`pip install 部品の名前` で入れます。	01 4. 道具をそろえる③：Python 3.12
`pip --version`	pip の版と、どの Python の pip かを表示します。仮想環境に入っていると、表示される場所に `venv` が入ります。	01 4. 道具をそろえる③：Python 3.12

コード	意味と使い方	初めて出てくる所
`python -m`	`python -m venv venv` の `-m` は「Python の中の道具（モジュール）を動かす」という指定。ここでは `venv`（仮想環境を作る道具）を動かして、`venv` という名前のフォルダに仮想環境を作ります。	01 8. バックエンド準備①：仮想環境
`.\venv\Scripts\Activate.ps1`	仮想環境（このアプリ専用の Python の部品の置き場）に入る PowerShell のスクリプト。入ると、行の頭に `(venv)` と出ます。「スクリプトの実行が無効」と出たら、`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` で許可します。	01 8. バックエンド準備①：仮想環境
`deactivate`	仮想環境から出るコマンド。行の頭の `(venv)` が消えます。	01 8. バックエンド準備①：仮想環境
`pip install`	Python の部品を入れます。`pip install 部品の名前` で 1 つ、`pip install -r requirements.txt` でファイルに書いた全部を入れます。	01 9. バックエンド準備②：requirements.txt...
`pip install -r`	`-r requirements.txt` は「このファイルに並んだ部品を全部入れる」という指定です。`requirements.txt` には `Django==4.2.7` のように、部品の名前と版が 1 行に 1 つずつ書いてあります。	01 9. バックエンド準備②：requirements.txt...
`pip list`	今の Python（仮想環境の中なら、その中）に入っている部品と版を一覧にします。	01 9. バックエンド準備②：requirements.txt...
`python -c`	`python -c "…"` は、後ろの文字を Python のプログラムとして、その場で動かします。ファイルを作るほどでない、短い確かめに使います。	01 10. バックエンド準備③：.env を作る
`pip uninstall`	入れた部品を消します。`pip uninstall 部品の名前` で、消してよいか聞かれたら `y` を押します。消したら `requirements.txt` からも、その行を消します。	11 12. 不要な依存を安全に整理する手順
`pip install --upgrade`	`pip install --upgrade pip` は、部品をいちばん新しい版にするコマンド。ここでは `pip` 自身を新しくしています。	15 7. 開発・運用の整理

Django の道具

コード	意味と使い方	初めて出てくる所
`.Env()`	django-environ（`.env` ファイルを読む部品）の道具。`env = environ.Env()` で、設定を読む係を作ります。	00 12. 実コード⑦：settings.py の主要設定を1...

コード	意味と使い方	初めて出てくる所
`.read_env()`	`env.read_env(ファイルの場所)` で、`.env` ファイルに書いた `名前=値` を読み込みます。秘密の値（パスワードなど）をプログラムに直接書かずに済みます。	00 12. 実コード⑦：settings.pyの主要設定を1…
`.bool()`	`env.bool('DEBUG')` は、`.env` の `DEBUG` の値を、`True` か `False`（真か偽）として読みます。文字の `"False"` を、本当の `False` に直してくれます。	00 12. 実コード⑦：settings.pyの主要設定を1…
`env()`	`env('SECRET_KEY')` は、`.env` に書いた値を文字として読みます。書いていないとエラーで止まるので、書き忘れにすぐ気付けます。	00 12. 実コード⑦：settings.pyの主要設定を1…
`python --settings`	`python manage.py test --settings=hozen_another.test_settings` の `--settings` は「この設定ファイルを使う」という指定です。テストの時だけ、試し用のデータベースなどの設定に切り替えます。	12 13. テストの実行コマンドと出力例
`python --noinput`	`python manage.py collectstatic --noinput` の `--noinput` は、「上書きしてよいか」などの質問をせずに進める、という指定です。人が答えられない GitHub Actions の中で使います。	15 7. 開発・運用の整理

Node.js

コード	意味と使い方	初めて出てくる所
`node`	Node.js（パソコンの上で JavaScript を動かす道具）を動かすコマンド。`node --version` で版を表示し、`node ファイル名.js` で、そのファイルを動かします。	01 5. 道具をそろえる④：Node.js v22
`node --version`	`node --version` は、入っている Node.js の版を表示します。このアプリでは `v22.…`（おすすめ）か `v20.…` なら大丈夫です。	01 5. 道具をそろえる④：Node.js v22
`npm`	Node.js の部品を入れる・動かす道具のコマンド。`npm --version` で版を確かめ、`npm install`（部品を入れる）・`npm start`（開発用のサーバーを動かす）・`npm run build`（本番用に作る）のように使います。このアプリでは、`frontend/` フォルダで打ちます。	01 5. 道具をそろえる④：Node.js v22
`npm --version`	`npm --version` は、Node.js の部品を入れる道具 npm の版を表示します。	01 5. 道具をそろえる④：Node.js v22

コード	意味と使い方	初めて出てくる所
`npm install`	`package.json` に書いてある部品（TypeScript など）を、まとめて入れるコマンド。入った部品は `node_modules` フォルダに入ります（Git には入れません）。最初の 1 回だけ打ちます。`npm install 部品の名前` なら、その部品を足します。	01 16. フロントエンド準備①：package.json 全…
`npm start`	`package.json` の `"scripts"` の `"start"` を動かします。このアプリでは、画面の開発用サーバーが起動し、ブラウザーで `http://localhost:3000` が開きます。止める時は `Ctrl` ＋ `C` です。	01 18. フロントエンドを起動する
`npm run`	`package.json` の `"scripts"` に書いたコマンドを動かすコマンド。このアプリでは `npm run build`（本番用のファイルを `frontend/build/` に作る）・`npm run test:run`（テストを 1 回流す）などです（`start` と `test` だけは `run` を省けます）。止める時は `Ctrl` ＋ `C` です。	01 19. 本番ビルド
`.env`	`process.env` は、プログラムが読める環境変数の入れ物です。このアプリの画面では、`process.env.REACT_APP_API_BASE_URL`（サーバーの住所）を読みます。React（react-scripts）で読めるのは、`REACT_APP_` で始まる名前だけです。	04 17. 実コード全解説：axios.ts 全8行
`npm test`	`package.json` の `"scripts"` の `"test"` を動かします。このアプリでは Vitest が起動し、ファイルを書き換えるたびにテストを流し直します（終わる時は `q`）。	09 4. テストの実行
`npm uninstall`	入れた部品を消し、`package.json` と `package-lock.json` からも消します。`npm uninstall 部品の名前`。	11 12. 不要な依存を安全に整理する手順
`npm ci`	`package-lock.json` に書かれたとおりの版で、部品を入れ直すコマンド（`node_modules` を一度消してから入れる）。`npm install` と違い、`package-lock.json` を書きかえないので、いつ・どこで入れても同じになります。	15 7. 開発・運用の整理

設定ファイル（JSON）

コード	意味と使い方	初めて出てくる所
`"name":`	`package.json` の `"name"` は、このプロジェクト（パッケージ）の名前です。	01 16. フロントエンド準備①：package.json 全…

コード	意味と使い方	初めて出てくる所
`"version":`	`package.json` の `"version"` は、このプログラム自身の版の番号です。	01 16. フロントエンド準備①：package.json 全…
`"private":`	`"private": true` は、「このパッケージは npm に公開しない」という印です。間違って公開するのを防ぎます。	01 16. フロントエンド準備①：package.json 全…
`"dependencies":`	`package.json` の `"dependencies"` は、アプリが動くのに必要な部品の一覧です。`"axios": "^1.11.0"` のように、部品の名前と版を書きます。`^` は「1.11.0 以上で、2.0.0 より前ならよい」という意味です。`npm install` で、ここに書いた部品が入ります。	01 16. フロントエンド準備①：package.json 全…
`"typescript":`	`"typescript": "^4.9.5"` は、TypeScript（型を確かめる道具）の部品と、その版です。`^` は「4.9.5 以上で、5.0.0 より前ならよい」という意味です。	01 16. フロントエンド準備①：package.json 全…
`"scripts":`	`"scripts"` には、`npm run 名前` で動かすコマンドを書きます。このアプリでは `"start": "react-scripts start"`（`npm start` で開発用のサーバー）・`"build": "react-scripts build"`（`npm run build` で本番用に作る）・`"test": "vitest"` などです。	01 16. フロントエンド準備①：package.json 全…
`"start":`	`"scripts"` の中の `"start"` は、`npm start` で動くコマンドです（ここでは `react-scripts start` ＝開発用のサーバーを起動）。	01 16. フロントエンド準備①：package.json 全…
`"build":`	`"scripts"` の中の `"build"` は、`npm run build` で動くコマンドです（本番用に、画面のファイルを `build/` フォルダにまとめる）。	01 16. フロントエンド準備①：package.json 全…
`"test":`	`"scripts"` の中の `"test"` は、`npm test` で動くコマンドです（ここでは `vitest` ＝ファイルを見張って、変わるたびにテストを流す）。	01 16. フロントエンド準備①：package.json 全…
`"test:ui":`	`"scripts"` の中の `"test:ui"` は、`npm run test:ui` で動くコマンドです（テストの結果をブラウザーの画面で見る）。	01 16. フロントエンド準備①：package.json 全…

コード	意味と使い方	初めて出てくる所
`"test:run":`	`"scripts"` の中の `"test:run"` は、`npm run test:run` で動くコマンドです（テストを 1 回だけ流して終わる）。	01 16. フロントエンド準備①：package.json 全…
`"eject":`	`"scripts"` の中の `"eject"` は、react-scripts が隠している設定を取り出すコマンドです。元に戻せないので、ふだんは使いません。	01 16. フロントエンド準備①：package.json 全…
`"eslintConfig":`	`package.json` の `"eslintConfig"` は、ESLint（コードの書き方を調べる道具）の設定です。	01 16. フロントエンド準備①：package.json 全…
`"browserslist":`	`package.json` の `"browserslist"` は、どの版のブラウザーまで動くように作るかの指定です。`"production"`（本番のビルド用）と `"development"`（開発用）に分けて書きます。	01 16. フロントエンド準備①：package.json 全…
`"devDependencies":`	`"devDependencies"` には、開発の時だけ使う部品と、その版を書きます。`npm install` で入ります。	01 16. フロントエンド準備①：package.json 全…
`"extends":`	「この設定を元にする」という指定。`"extends": ["react-app"]` は「React の決まった ESLint の設定を土台にする」という意味です。	11 5. package.json の scripts / e…
`"production":`	`"browserslist"` の中の、本番用（`npm run build` の時）の対象のブラウザーの指定です。	11 5. package.json の scripts / e…
`"development":`	`"browserslist"` の中の、開発用（`npm start` の時）の対象のブラウザーの指定です。	11 5. package.json の scripts / e…

psql

コード	意味と使い方	初めて出てくる所
`psql`	PostgreSQL に、コマンドでつないで SQL を打つ道具。`psql -h 127.0.0.1 -U postgres -d データベース名` でつなぎます（`-h` は相手の住所、`-U` は利用者の名前、`-d` はデータベース）。`\q` で抜けます。	01 6. 道具をそろえる⑤：PostgreSQL
`psql --version`	`psql --version` は、psql（PostgreSQL につなぐ道具）の版を表示します。	01 6. 道具をそろえる⑤：PostgreSQL
`psql -U`	`-U postgres` は、どの利用者（ロール）としてつなぐかです。パスワードを聞かれたら、その利用者のパスワードを入れます。	01 12. バックエンド準備④：データベースを作ってmigr…
`psql -c`	`-c "SELECT …;"` は、つないで、その SQL を 1 回だけ動かして、すぐに終わる指定です。確かめを 1 行で済ませる時に使います。	01 12. バックエンド準備④：データベースを作ってmigr…

SQL

コード	意味と使い方	初めて出てくる所
`CREATE`	`CREATE ROLE`（ユーザーを作る）・`CREATE DATABASE`（データベースを作る）・`CREATE TABLE`（表を作る）のように、何かを新しく作る SQL の命令です。	01 12. バックエンド準備④：データベースを作ってmigr…
`DATABASE`	`CREATE DATABASE 名前` で、データベースを作ります。	01 12. バックエンド準備④：データベースを作ってmigr…
`UPDATE`	`UPDATE 表 SET 列 = 値 WHERE 条件` で、行の中身を書きかえる SQL の命令。`WHERE` を忘れると、全部の行が変わります。	06 18. 実コード全解説⑤：signals.py の差分計算

コード	意味と使い方	初めて出てくる所
`DELETE`	`DELETE FROM 表 WHERE 条件` で、表の行を消す SQL の命令。`WHERE` を忘れると、全部の行が消えます。`ON DELETE CASCADE` の中では「消された時」という意味です。	06 18. 実コード全解説⑤：signals.py の差分計算

Git

コード	意味と使い方	初めて出てくる所
`git`	ファイルの変更の記録を残す道具（Git）の本体のコマンド。`git --version` で版を確かめられます。`git status`・`git add`・`git commit` のように、後ろに「何をするか」を付けて使います。	01 3. 道具をそろえる②：Git
`git --version`	`git --version` は、入っている Git の版を表示します。	01 3. 道具をそろえる②：Git
`git config`	Git の設定を見る・変えるコマンド。`git config --global user.name "名前"` で、記録に残す名前を決めます。`--list` で、今の設定を一覧で見られます。	01 3. 道具をそろえる②：Git
`git config --global`	`--global` は、自分（このユーザー）の、全部のリポジトリに効く設定を対象にする指定です。`git config --global user.name "名前"` のように、最初に 1 回だけ設定します。	01 3. 道具をそろえる②：Git
`git clone`	GitHub などにあるリポジトリ（プログラムの置き場所）を、丸ごと自分のパソコンに写してきます。`git clone <URL>` で、URL の最後の名前のフォルダができます。	01 7. ソースコードを手元に持ってくる
`git status`	今の状態（変わったファイル・記録に入れる予定のファイル・Git がまだ知らないファイル）を表示します。迷ったら、まずこれを打ちます。	02 3. このプロジェクトの Git の状態を見てみる
`git remote`	つながっている GitHub など（リモート）の名前を表示します。ふつうは `origin` だけです。	02 3. このプロジェクトの Git の状態を見てみる
`git remote -v`	リモートの名前と、その URL を表示します（`-v` は「くわしく」）。取ってくる先（fetch）と送る先（push）の 2 行が出ます。	02 3. このプロジェクトの Git の状態を見てみる
`git branch`	ブランチ（作業の枝）の一覧を出します。今いるブランチには `*` が付きます。`git branch 名前` で新しいブランチを作ります。	02 3. このプロジェクトの Git の状態を見てみる

コード	意味と使い方	初めて出てくる所
`git branch -a`	自分のパソコンのブランチに加えて、GitHub 側のブランチ（`remotes/origin/…`）も全部表示します（`-a` は all）。	02 3. このプロジェクトの Git の状態を見てみる
`git status -s`	`git status` を短く（short）表示します。1 行に 1 つのファイルで、頭の `M` は変更、`??` はまだ Git が知らないファイル、`A` は追加の予約済みです。	02 4. git status — 今どうなっているか
`git diff`	まだ記録していない変更の中身を見るコマンド。`-` で始まる行が前の内容、`+` で始まる行が新しい内容です。	02 5. git diff — 何をどう変えたか
`git diff --staged`	`git add` で「次のコミットに入れる」と予約した変更の中身を表示します。コミットする前の最後の確かめに使います。	02 5. git diff — 何をどう変えたか
`git add`	次の記録（コミット）に入れるファイルを選ぶコマンド。`git add ファイル名`、全部なら `git add .` です。選んだだけで、まだ記録はされていません。	02 6. git add — 記録の準備
`git restore`	保存したけれど記録（コミット）していない変更を捨てて、最後の記録の状態に戻すコマンド。`git restore frontend/src/MainPage/Help.tsx`。捨てた変更は戻せないので、`git diff` で確かめてから使います。	02 6. git add — 記録の準備
`git restore --staged`	`git add` の予約を取り消します（ファイルの中身は変わりません）。間違えて `add` したファイルを外す時に使います。	02 6. git add — 記録の準備
`git commit`	選んだ変更を、1 つの記録（コミット）として残すコマンド。`git commit -m "何をしたか"` の `-m` の後ろに、変更の説明を書きます。	02 7. git commit — 記録を確定する
`git commit -m`	`git commit -m "説明"` の `-m` は、コミットの説明文を、その場で書く指定です。付けないと、説明を書くための画面（エディター）が開きます。	02 7. git commit — 記録を確定する
`git log`	これまでの記録（コミット）の一覧を、新しい順に出すコマンド。`--oneline` で 1 行ずつ短く、`--format="…"` で表示の形を自分で決められます。	02 8. git log — 過去の記録を読む
`git log --oneline`	`--oneline` は、1 件のコミットを 1 行（短い番号と、説明の 1 行目）で表示します。	02 8. git log — 過去の記録を読む
`git log --graph`	コミットの歴史を、ブランチの枝分かれと合流が分かる線の絵で表示します。	02 8. git log — 過去の記録を読む

コード	意味と使い方	初めて出てくる所
`git log --all`	今いるブランチだけでなく、全部のブランチのコミットを表示します。`git log --oneline --graph --all` と組み合わせると、全体の枝の様子が分かります。	02 8. git log — 過去の記録を読む
`git show`	1 つの記録（コミット）の中身を見るコマンド。`git show 9156f15` で、そのコミットの説明と、変わった所（`-` が前、`+` が後）を表示します。`git show コミット:ファイル` で、その時のファイルの中身も見られます（一度記録した秘密のファイルも、こうして見えてしまいます）。	02 8. git log — 過去の記録を読む
`git push`	自分のパソコンのコミットを、GitHub（リモート）へ送ります。`git push origin master` は「`origin` の `master` へ送る」。	02 9. リモートとの同期：push / pull / fet…
`git pull`	GitHub の新しいコミットを取ってきて、今のブランチに取り込みます（`git fetch` と `git merge` を続けてやるのと同じ）。作業を始める前に、まず `git pull` します。	02 9. リモートとの同期：push / pull / fet…
`git fetch`	GitHub の新しいコミットを取ってくるだけで、自分の作業中のファイルには混ぜません。何が増えたかを先に見たい時に使います。	02 9. リモートとの同期：push / pull / fet…
`git switch`	ブランチを切り替えます。`git switch master` で `master` へ移ります。	02 10. ブランチ — 作業の枝分かれ
`git switch -c`	新しいブランチを作って、そこへ切り替えます（`-c` は create＝作る）。`git switch -c help-fix`。	02 10. ブランチ — 作業の枝分かれ
`git merge`	ほかのブランチの変更を、今いるブランチに取り込みます。`master` にいる時に `git merge help-fix` で、`help-fix` の変更が `master` に入ります。	02 11. merge
`git branch -d`	ブランチを消します。取り込み（マージ）が済んでいないブランチは、消さずに止まって教えてくれます。	02 11. merge
`git rm`	ファイルを、Git の記録から外すコマンド。`git rm --cached .env` は、ファイルそのものは残したまま、記録からだけ外します（間違えて記録した秘密のファイルを外す時に使います）。	02 14. .gitignore — 追跡しないファイルの指定
`git rm --cached`	`git rm --cached .env` は、ファイルは消さずに、Git の管理からだけ外します。うっかり入れてしまった秘密のファイルを、次のコミットから外す時に使います。	02 14. .gitignore — 追跡しないファイルの指定
`git commit --amend`	直前のコミットを作り直します。`git commit --amend -m "正しいメッセージ"` でメッセージを直せます。もう `push` したコミットには使いません（GitHub 側とずれます）。	02 16. 復旧コマンドと「やってしまった」対処集

コード	意味と使い方	初めて出てくる所
`git reset`	今いるブランチの位置を、前のコミットへ戻します。`HEAD~1` は「1 つ前のコミット」です。	02 16. 復旧コマンドと「やってしまった」対処集
`git reset --soft`	コミットだけを取り消し、変更は `add` 済みのまま残します。「コミットをやり直したい」時に使います。	02 16. 復旧コマンドと「やってしまった」対処集
`git reset --mixed`	コミットと `add` を取り消し、変更はファイルに残します（何も付けない `git reset` と同じ）。	02 16. 復旧コマンドと「やってしまった」対処集
`git reset --hard`	コミットも、ファイルの変更も、全部消して戻します。消した変更は元に戻せないことがあるので、使う前に必ず `git status` で確かめます。	02 16. 復旧コマンドと「やってしまった」対処集
`git reflog`	`HEAD`（今いる位置）が動いた記録の一覧。`git reset --hard` で消してしまったコミットも、ここに出る番号から取り戻せることがあります。	02 16. 復旧コマンドと「やってしまった」対処集
`git add --renormalize`	`git add --renormalize .` は、記録されているファイルを、今の `.gitattributes` の決まり（改行の扱いなど）で記録し直すコマンド。`.gitattributes` を変えた後に使います。	15 6. 不具合
`git rm -r`	`git rm` の `-r` は、フォルダの中を全部、という指定です。`git rm -r --cached staticfiles` は、`staticfiles` フォルダの中を全部、Git の記録から外します（手元のファイルは残る）。	15 7. 開発・運用の整理

HTML

コード	意味と使い方	初めて出てくる所
`<form>`	入力欄とボタンをまとめて、送る（サーバーへ届ける）ためのタグ。`action` に送り先、`method="post"` に送り方を書きます。	00 5. 実コード①：Login.tsx
`<label>`	入力欄の名前を表すタグ。`<label for="numberInput">従業員番号</label>` の `for` と、入力欄の `id` を同じにすると、名前を押しても入力欄が選ばれ、読み上げソフトにも伝わります（React では `for` の代わりに `htmlFor` と書きます）。	00 5. 実コード①：Login.tsx
`<input>`	1 行の入力欄のタグ。`type` で種類（`text`・`email`・`password`・`hidden` など）を、`name` で送る時の名前を決めます。閉じタグはありません。	00 5. 実コード①：Login.tsx

コード	意味と使い方	初めて出てくる所
`<div>`	特別な意味を持たない「箱」のタグ。CSS で見た目や並べ方を決めるために、ほかのタグをまとめる時に使います。	03 0. 前提知識：ブラウザが画面を作るしくみ
`<h1>`	いちばん大きな見出し（heading 1）のタグ。数が大きいほど小さな見出しになります（`<h2>` 〜 `<h6>`）。`<h1>` は、1 ページに 1 つにします。	03 0. 前提知識：ブラウザが画面を作るしくみ
`<p>`	段落（paragraph）のタグ。ひとまとまりの文を囲みます。段落と段落の間には、自動で少し間が空きます。	03 0. 前提知識：ブラウザが画面を作るしくみ
`<!-- -->`	HTML では、`<!--` から `-->` までがコメントで、画面には出ません。	03 1. HTMLの基本
`<img>`	画像を表示するタグ。`<img src="logo.png" alt="ロゴ">` のように、`src` に画像の場所、`alt` に画像の説明を書きます。中身を持たないので、閉じタグ（`</img>`）は書きません。	03 1. HTMLの基本
`<input>`	1 行の入力欄のタグ。`type` で種類（`text`・`email`・`password`・`hidden` など）を、`name` で送る時の名前を決めます。閉じタグはありません。	03 1. HTMLの基本
` `	改行のタグ。閉じタグはありません。段落を分けたい時は、` ` を並べるより `<p>` を分けます。	03 1. HTMLの基本
`<hr>`	横線（水平線）を引くタグ。話の区切りに使います。中身を持たないので、閉じタグは書きません。	03 1. HTMLの基本
`src=`	`src` は、読み込むファイル（画像・スクリプト）の場所です。	03 1. HTMLの基本
`alt=`	`<img alt="ロゴ">` の `alt` は、画像の代わりの説明の文。画像が表示できない時や、読み上げソフトで読む時に使われます。	03 1. HTMLの基本
`type=`	`<input type="…">` の `type` は、入力欄の種類です。`text`（文字）・`email`・`password`（伏せ字）・`number`・`date`・`checkbox`（チェック）・`file`（ファイル）・`search` などがあります。`<button type="submit">` は、押すとフォームを送るボタンです。	03 1. HTMLの基本
`<a>`	リンクのタグ。`<a href="URL">文字</a>` で、押すとその URL へ移ります。	03 1. HTMLの基本
`href=`	`href` は、リンク先や、読み込むファイルの場所（URL）です。`<a href="/help">ヘルプ</a>` は、押すと `/help` へ移るリンクです。	03 1. HTMLの基本

コード	意味と使い方	初めて出てくる所
`target=`	`<a href="/help" target="_blank">` の `target="_blank"` は「リンク先を新しいタブで開く」という指定です。	03 1. HTMLの基本
`class=`	`class` は、タグに付ける「グループの名前（名札）」です。CSS で `.blue_button { … }` と書くと、`class="blue_button"` の物全部に、同じ見た目が付きます。空白で区切って、いくつも付けられます。	03 1. HTMLの基本
`<h6>`	いちばん小さい見出しのタグ。見出しは `<h1>`（いちばん大きい）から `<h6>`（いちばん小さい）まで 6 段階あります。	03 1. HTMLの基本
`<span>`	文の一部分を囲む、特別な意味の無いタグ。CSS で、その部分だけ色を変える時などに使います（`<div>` と違い、改行しません）。	03 1. HTMLの基本
`<ul>`	順番の無い並び（点の付いた並び）のタグ。中に `<li>` を並べます。	03 1. HTMLの基本
`<li>`	並び（`<ul>`・`<ol>`）の 1 つのタグ。	03 1. HTMLの基本
`<ol>`	番号の付いた並びのタグ。中に `<li>` を並べると、1. 2. 3. と番号が付きます（手順など）。	03 1. HTMLの基本
`<table>`	表のタグ。中に `<thead>`（見出しの行）と `<tbody>`（中身の行）を入れ、行は `<tr>`、ますは `<th>`（見出し）か `<td>`（中身）で作ります。	03 1. HTMLの基本
`<thead>`	表の見出しの行をまとめるタグ。	03 1. HTMLの基本
`<tbody>`	表の中身の行をまとめるタグ。	03 1. HTMLの基本
`<tr>`	表の 1 行（table row）のタグ。中に `<th>` や `<td>` を並べます。	03 1. HTMLの基本
`<th>`	表の見出しのます（table header）のタグ。ふつうは太字で表示されます。	03 1. HTMLの基本
`<td>`	表の中身のます（table data）のタグ。	03 1. HTMLの基本
`<nav>`	メニュー（ほかのページへのリンクのまとまり）を表すタグ。	03 1. HTMLの基本
`<form>`	入力欄とボタンをまとめて、送る（サーバーへ届ける）ためのタグ。`action` に送り先、`method="post"` に送り方を書きます。	03 1. HTMLの基本

コード	意味と使い方	初めて出てくる所
`<label>`	入力欄の名前を表すタグ。`<label for="numberInput">従業員番号</label>` の `for` と、入力欄の `id` を同じにすると、名前を押しても入力欄が選ばれ、読み上げソフトにも伝わります（React では `for` の代わりに `htmlFor` と書きます）。	03 1. HTMLの基本
`<button>`	ボタンのタグ。フォームの中の `<button type="submit">送る</button>` は、押すとフォームを送ります。`type="button"` にすると、送らない、ふつうのボタンになります。	03 1. HTMLの基本
`<select>`	選ぶ欄（プルダウン）のタグ。中に `<option value="値">表示する文字</option>` を並べます。`selected` を付けた物が、最初に選ばれます。	03 1. HTMLの基本
`<option>`	`<select>`（選ぶ欄）の中の、選べる 1 つ。`<option value="new">新しく作る</option>` の `value` が送られる値、間の文字が画面に出る文字です。`selected` で最初に選ばれます。	03 1. HTMLの基本
`<textarea>`	何行も書ける入力欄のタグ。`<textarea name="detail" rows="4"></textarea>` の `rows` は、見えている行の数です。最初から入れておく文字は、タグの間に書きます。	03 1. HTMLの基本
`for=`	`<label for="q">` の `for` は、この説明の文字が、どの入力欄（`id="q"`）のための物かを結び付けます。文字を押しても、その欄に入力できるようになり、読み上げソフトも欄の名前を読めます。	03 2. HTMLフォーム
`id=`	`id` は、ページの中で 1 つだけの名前です。`<label for="title">` で欄と結び付けたり、`#title` のリンクでその場所へ飛んだりできます。同じ `id` を 2 つ付けてはいけません。	03 2. HTMLフォーム
`value=`	`value` は、入力欄の最初の値や、選び肢（`<option>`）を選んだ時に送られる値です。	03 2. HTMLフォーム
`rows=`	`<textarea rows="6">` の `rows` は、見える行の数です。	03 2. HTMLフォーム

CSS

コード	意味と使い方	初めて出てくる所
`color:`	文字の色を決める項目。`color: red;` や `color: #333;`、`color: var(--main-color);`（変数の色）のように書きます。	03 4. CSSの基本文法
`font-size:`	CSS で、文字の大きさを決める項目。	03 4. CSSの基本文法

コード	意味と使い方	初めて出てくる所
`/* */`	CSS では、`/*` から `*/` までがコメントです。	03 4. CSSの基本文法
`数px`	CSS の `16px` の `px` は、画面の点（ピクセル）の数で表す大きさの単位です。	03 4. CSSの基本文法
`box-sizing:`	CSS で、箱の幅に枠線と内側の余白を含めるかを決める項目。`box-sizing: border-box;` にすると、`width` の中に `padding` と `border` が含まれ、大きさの計算が分かりやすくなります。	03 5. ボックスモデル
`*`	CSS の `*` は、「全部のタグ」を選ぶ書き方です。`* { box-sizing: border-box; }` は、全部のタグの幅の数え方をそろえます。	03 5. ボックスモデル
`margin:`	箱の外側の余白を決める項目。`margin: 12px;` は上下左右に 12px。2 つ書くと「上下 左右」、4 つ書くと「上 右 下 左」の順です。	03 5. ボックスモデル
`margin-top:`	CSS で、箱の上の外側の余白を決める項目。	03 5. ボックスモデル
`rgb()`	色を「赤・緑・青」の強さ（0〜255）で書く書き方。`rgb(0, 123, 255)` は青っぽい色です。	03 6. 単位と色
`rgba()`	`rgb()` に、透けぐあい（0＝透明〜1＝不透明）を足した書き方。`rgba(0, 0, 0, 0.1)` は、薄く透けた黒（影などに使う）です。	03 6. 単位と色
`#rrggbb`	CSS の `#0f172a` のような `#` と 6 けた（または 3 けた）の 16 進数は、色の書き方です。2 けたずつ、赤・緑・青の強さ（00〜ff）を表します。`#ffffff` は白、`#000000` は黒です。	03 6. 単位と色
`display:`	箱の並べ方を決める項目。`display: flex;` は「中身を横一列に並べる」、`display: grid;` は「ます目に並べる」、`display: none;` は「表示しない」です。	03 7. Flexbox
`flex-direction:`	`display: flex` の中身を並べる向きを決める項目。`row` は横（最初の値）、`column` は縦に並べます。	03 7. Flexbox
`justify-content:`	`display: flex` の中身を、並ぶ向きにどう配るかを決める項目。`space-between` は「両端に寄せて、間を均等に空ける」。	03 7. Flexbox
`align-items:`	`display: flex` の中身を、上下（横に並べた時の縦）のどこにそろえるかを決める項目。`center` で真ん中にそろえます。	03 7. Flexbox
`gap:`	`flex` や `grid` の中身どうしの間の幅を決める項目。`gap: 12px 16px;` は「行の間 12px、列の間 16px」。	03 7. Flexbox
`flex-wrap:`	`display: flex` の中身が入りきらない時に、次の行へ折り返すかを決める項目。`wrap` で折り返します。	03 7. Flexbox

コード	意味と使い方	初めて出てくる所
`.クラス名`	CSS の `.blue_button { … }` のように `.` で始まる名前は、「`class="blue_button"` の付いたタグ全部」を選ぶ書き方（クラスセレクター）です。`{ }` の中の見た目が、その全部に付きます。	03 7. Flexbox
`button:`	`button:hover` は、「マウスを乗せている時のボタン」を指す CSS の書き方（`:hover` は乗せている時の状態）。	03 8. 擬似クラスと擬似要素
`:hover`	CSS の `button:hover` の `:hover` は、「マウスが乗っている間だけ」の見た目です。`:focus` は「キーボードで選ばれている間」、`:disabled` は「使えない時」です。	03 8. 擬似クラスと擬似要素
`:focus`	入力欄などが「今、入力の場所になっている（カーソルがある）」時だけ当てるスタイル。`input:focus { … }`。	03 8. 擬似クラスと擬似要素
`:nth-child`	「何番目の子か」で選ぶ書き方。`tr:nth-child(even)` は偶数番目の行、`odd` なら奇数番目の行です。表を 1 行おきに色分けする時に使います。	03 8. 擬似クラスと擬似要素
`:disabled`	押せない（使えない）状態のボタンや入力欄にだけ当てるスタイル。`button:disabled { opacity: 0.5; }`。	03 8. 擬似クラスと擬似要素
`content:`	`::before` や `::after` で足す中身。`content: "⚠️ ";` は文字を足し、`content: "";` は中身の無い飾り（形や色だけ）を作ります。	03 8. 擬似クラスと擬似要素
`::before`	部品の中身の前に、CSS で飾りの文字や形を足す書き方。`content: "⚠️ ";` で足す物を決めます。	03 8. 擬似クラスと擬似要素
`::after`	部品の中身の後ろに、CSS で飾りの文字や形を足す書き方。`content:` とセットで使います。	03 8. 擬似クラスと擬似要素
`background-color:`	背景の色。`background-color: #007bff;` は青の背景です。	03 9. トランジションとアニメーション
`transition:`	スタイルが変わる時に、少しずつ変える（なめらかに動かす）指定。`transition: background-color 0.3s ease;` は「背景の色を 0.3 秒かけて変える」。	03 9. トランジションとアニメーション
`数s`	`0.3s` の `s` は秒（second）です。`transition` や `animation` の長さに使います。	03 9. トランジションとアニメーション
`opacity:`	透明さを決める項目。`1` で不透明、`0` で見えない。`0.85` は少しだけ透ける（ボタンにマウスを乗せた時の合図など）。	03 9. トランジションとアニメーション
`transform:`	部品を動かす・回す・大きくするなど、形を変える指定。`transform: translateY(50px);` は 50px 下へずらします。	03 9. トランジションとアニメーション

コード	意味と使い方	初めて出てくる所
`animation:`	`@keyframes` で決めた動きを部品に付けます。`animation: fadeInUp 1s ease-out;` は「`fadeInUp` の動きを 1 秒で、終わりをゆっくり」。	03 9. トランジションとアニメーション
`@keyframes`	動き（アニメーション）の流れを名前を付けて決める書き方。`@keyframes fadeInUp { from { … } to { … } }` で、始め（from）と終わり（to）の様子を書きます。	03 9. トランジションとアニメーション
`translateY()`	縦（Y の向き）にずらす書き方。`translateY(50px)` は下へ 50px、マイナスなら上へずらします。	03 9. トランジションとアニメーション
`max-width:`	`@media (max-width: 640px) { … }` は、「画面の幅が 640px 以下の時だけ、中の決まりを使う」という書き方（スマホ向けの見た目）。箱に書く `max-width: 760px;` は、「幅の上限」です。	03 10. メディアクエリ
`width:`	CSS で、箱の幅を決める項目。	03 10. メディアクエリ
`min-width:`	いちばん小さい時の幅。これより細くはなりません。`unset` は「指定しない」に戻します。	03 10. メディアクエリ
`@media`	CSS の `@media (max-width: 640px) { … }` は、「画面の幅が 640px 以下の時だけ」の見た目を書く所です（メディアクエリ）。スマホでは 1 列にする、などに使います。	03 10. メディアクエリ
`(max-width)`	`@media` の `(max-width: 768px)` は、「画面の幅が 768px 以下」（スマートフォンや小さい画面）という条件です。`(min-width: …)` なら「以上」です。	03 10. メディアクエリ
`padding:`	箱の内側の余白（枠と中身の間）を決める項目。書き方は `margin` と同じです。	03 11. 実コード全解説：global.css 全342行を…
`height:`	高さ。`height: 100%;` は親の高さいっぱいです。	03 11. 実コード全解説：global.css 全342行を…
`font-family:`	CSS で、文字の書体（フォント）を決める項目。`font-family: sans-serif;` は「飾りの無い書体」。	03 11. 実コード全解説：global.css 全342行を…
`A, B`	CSS の `button, .btn { … }` のように `,` で区切ると、「`button` と `.btn` の両方」に、同じ見た目を付けます。	03 11. 実コード全解説：global.css 全342行を…

コード	意味と使い方	初めて出てくる所
`数%`	CSS の `100%` は、「入れ物（親）の大きさの何パーセントか」で表す単位です。	03 11. 実コード全解説：global.css 全342行を…
`border:`	CSS で、箱の枠線（太さ・線の種類・色）をまとめて決める項目。`border: 1px solid #e2e8f0;`。	03 11. 実コード全解説：global.css 全342行を…
`background:`	CSS で、背景（色や画像）を決める項目。`background: var(--color-bg);` は、色の変数の色にします。	03 11. 実コード全解説：global.css 全342行を…
`outline:`	部品の外側の線（入力の場所になった時に出る枠など）。`outline: none;` で消します。	03 11. 実コード全解説：global.css 全342行を…
`list-style:`	CSS で、並び（`ul`・`ol`）の点や番号の形を決める項目。`none` で消せます。	03 11. 実コード全解説：global.css 全342行を…
`text-decoration:`	CSS で、文字の下線などを決める項目。`none` で、リンクの下線を消せます。	03 11. 実コード全解説：global.css 全342行を…
`:root`	CSS の `:root` は、ページ全体（いちばん外側の `<html>`）を選ぶ書き方です。ページのどこからでも使う色などの変数（`--color-text`）を、ここに書きます。	03 11. 実コード全解説：global.css 全342行を…
`--名前`	CSS の `--color-text: #0f172a;` のように `--` で始まる名前は、自分で決める変数（カスタムプロパティ）です。`color: var(--color-text);` で使います。色を 1 か所で決めておけば、変える時も 1 か所で済みます。	03 11. 実コード全解説：global.css 全342行を…
`clamp()`	「いちばん小さい値・ふつうの値・いちばん大きい値」を並べて、その間に収める書き方。`clamp(1.5rem, 1.5rem + 3vw, 4rem)` は「画面の幅に合わせて大きくなるが、1.5rem より小さく、4rem より大きくはならない」。	03 11. 実コード全解説：global.css 全342行を…
`数rem`	CSS の `1.5rem` の `rem` は、「ページの基本の文字の大きさ（ふつう 16px）の何倍か」で表す単位です。利用者が文字を大きくする設定にすると、一緒に大きくなるので、読みやすさのために、文字や余白によく使います。	03 11. 実コード全解説：global.css 全342行を…

コード	意味と使い方	初めて出てくる所
`数vw`	`3vw` の `vw` は「画面の幅の 1%」です。`100vw` で画面の幅いっぱいになります。	03 11. 実コード全解説：global.css 全342行を…
`min-height:`	いちばん小さい時の高さ。`min-height: 100vh;` は「少なくとも画面の高さいっぱい」です。	03 11. 実コード全解説：global.css 全342行を…
`overflow-x:`	横にはみ出した時の扱い。`overflow-x: auto;` は「はみ出したら横のスクロールバーを出す」。	03 11. 実コード全解説：global.css 全342行を…
`white-space:`	CSS で、空白や改行をどう表示するかを決める項目。`pre-wrap` は「改行と空白はそのまま、はみ出す所は折り返す」。	03 11. 実コード全解説：global.css 全342行を…
`margin-bottom:`	CSS で、箱の下の外側の余白を決める項目。	03 11. 実コード全解説：global.css 全342行を…
`text-align:`	CSS で、文字の左右のそろえ方を決める項目。`center` で中央。	03 11. 実コード全解説：global.css 全342行を…
`数vh`	`100vh` の `vh` は「画面の高さの 1%」です。`100vh` で画面の高さいっぱいになります。	03 11. 実コード全解説：global.css 全342行を…
`var()`	CSS の `var(--color-text)` は、`--` で始まる変数の値を使う書き方です。	03 11. 実コード全解説：global.css 全342行を…
`border-radius:`	CSS で、箱の角の丸みを決める項目。	03 11. 実コード全解説：global.css 全342行を…
`box-shadow:`	部品に影を付けます。`box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);` は「横 0・下へ 4px・ぼかし 6px の、薄い黒の影」。	03 11. 実コード全解説：global.css 全342行を…
`border-collapse:`	表のマスの線の重なり方。`border-collapse: collapse;` で、となりのマスの線を 1 本にまとめます。	03 11. 実コード全解説：global.css 全342行を…

コード	意味と使い方	初めて出てくる所
`font-weight:`	CSS で、文字の太さを決める項目。`700` や `bold` で太字、`800` はもっと太い字です。	03 11. 実コード全解説：global.css 全342行を…
`text-shadow:`	文字に影を付けます。向きを変えた影をいくつも重ねると、文字のふち取りになります。	03 11. 実コード全解説：global.css 全342行を…
`cursor:`	CSS で、マウスを乗せた時の矢印の形を決める項目。`pointer` は、押せる物の「指の形」です。	03 11. 実コード全解説：global.css 全342行を…
`writing-mode:`	文字を書く向き。`horizontal-tb` は横書き（上から下へ行が進む）、`vertical-rl` は縦書きです。	03 11. 実コード全解説：global.css 全342行を…
`overflow-wrap:`	CSS で、長い単語（URL など）が箱からはみ出す時に、途中で折り返すかを決める項目。`anywhere` で、どこでも折り返せます。	03 11. 実コード全解説：global.css 全342行を…
`word-break:`	長い言葉の折り返し方。`word-break: break-all;` は、英語の単語の途中でも折り返して、はみ出さないようにします。	03 11. 実コード全解説：global.css 全342行を…
`[属性]`	属性（タグに書いた `type="date"` など）で選ぶ書き方（属性セレクタ）。`input[type="date"]` は日付の入力欄、`div[role="alert"]` は `role="alert"` の `<div>` を選びます。	03 11. 実コード全解説：global.css 全342行を…
`A B`	CSS の `.search-bar input` のように、空白で区切って並べると、「`.search-bar` の**中にある** `input` 」を選びます（子孫セレクター）。	03 11. 実コード全解説：global.css 全342行を…
`position:`	置き方の決まり。`relative`（ふつうの位置から少しずらす）、`absolute`（近くの `relative` の親を基準に、好きな位置へ置く）、`fixed`（画面に固定して、スクロールしても動かない）などがあります。	03 11. 実コード全解説：global.css 全342行を…
`webkit-appearance:`	`-webkit-appearance: none;` は、ブラウザーが決めている入力欄などの見た目を消して、自分の CSS で見た目を作れるようにします（`-webkit-` は Chrome・Edge・Safari 向けの印）。	03 11. 実コード全解説：global.css 全342行を…
`top:`	`position` を決めた部品の、上からの位置。`top: 0;` で上の端に付きます。	03 11. 実コード全解説：global.css 全342行を…

コード	意味と使い方	初めて出てくる所
`left:`	`position` を決めた部品の、左からの位置。	03 11. 実コード全解説：global.css 全342行を…
`z-index:`	重なった時の上下の順番。数が大きいほど上（手前）に出ます。`position` を決めた部品で効きます。	03 11. 実コード全解説：global.css 全342行を…
`::-webkit-calendar-picker-indicator`	日付の入力欄の右にある、カレンダーの印の部分（Chrome・Edge など）。ここに CSS を当てて、印を消したり、欄全体を押せるようにしたりします。	03 11. 実コード全解説：global.css 全342行を…
`::-webkit-inner-spin-button`	数や日付の入力欄の、上下の小さな矢印のボタンの部分（Chrome・Edge など）。`display: none;` で消せます。	03 11. 実コード全解説：global.css 全342行を…
`::-webkit-clear-button`	日付の入力欄の、中身を消す × ボタンの部分（古い Chrome など）。`display: none;` で消せます。	03 11. 実コード全解説：global.css 全342行を…
`max-height:`	いちばん大きい時の高さ。これより高くはなりません。	03 12. 実コード全解説：Login.module.css …
`overflow-y:`	縦にはみ出した時の扱い。`overflow-y: auto;` は「はみ出したら縦のスクロールバーを出す」。	03 12. 実コード全解説：Login.module.css …
`webkit-overflow-scrolling:`	`-webkit-overflow-scrolling: touch;` は、iPhone などで、はみ出した部分を指でなめらかにスクロールできるようにする古い指定です。	03 12. 実コード全解説：Login.module.css …
`object-fit:`	画像を枠に入れる時の合わせ方。`contain` は「縦横の比を保って、全部が見えるように収める」、`cover` は「枠いっぱいに広げて、はみ出しは切る」。	03 12. 実コード全解説：Login.module.css …
`min()`	いくつかの値の中で、いちばん小さい物を使う書き方。`width: min(300px, 80%);` は「300px か 80% の、小さい方」。	03 12. 実コード全解説：Login.module.css …

コード	意味と使い方	初めて出てくる所
`border-color:`	CSS で、枠線の色だけを決める項目。	03 12. 実コード全解説：Login.module.css …
`calc()`	計算した値を使う書き方。`width: calc(100% - 40px);` は「親の幅から 40px 引いた幅」。`+` と `-` の前後には空白が要ります。	03 12. 実コード全解説：Login.module.css …
`bottom:`	`position` を決めた部品の、下からの位置。`bottom: 30px;` で下から 30px の所に置きます。	03 12. 実コード全解説：Login.module.css …
`right:`	`position` を決めた部品の、右からの位置。	03 12. 実コード全解説：Login.module.css …
`rotate()`	回す書き方。`rotate(45deg)` は右回りに 45 度回します。	03 12. 実コード全解説：Login.module.css …
`数deg`	`45deg` の `deg` は角度の「度」（degree）です。	03 12. 実コード全解説：Login.module.css …
`flex:`	`display: flex` の中の部品の伸び縮み。`flex: 0 0 auto;` は「伸びない・縮まない・大きさは中身に合わせる」。	03 13. 実コード追加解説：KosuList / MainM…
`数em`	`1em` は「その部品の文字の大きさ 1 つ分」です。文字の大きさに合わせて伸び縮みします。	03 13. 実コード追加解説：KosuList / MainM…
`!important`	「このスタイルをいちばん強くする」印。`display: none !important;` は、ほかの指定より必ず勝ちます。多く使うと、どれが効いているか分かりにくくなるので、必要な所だけに使います。	03 13. 実コード追加解説：KosuList / MainM…

コード	意味と使い方	初めて出てくる所
`clip-path:`	部品を好きな形に切り抜きます。`clip-path: polygon(0% 0%, 100% 50%, 0% 100%);` は 3 つの点を結んだ三角形（右向きの矢印の形）です。	03 13. 実コード追加解説：KosuList / MainM…
`padding-left:`	左側の内側の余白。	03 13. 実コード追加解説：KosuList / MainM…

JavaScript・TypeScript

コード	意味と使い方	初めて出てくる所
`import`	ほかのファイルや部品から、関数などを読み込む書き方。`import React, { useState } from "react";` は、React の本体（`React`）と、その中の `useState` を使えるようにします。`{ }` で囲むのは「中の 1 つを名前で選んで取る」、囲まないのは「そのファイルの代表（`export default` された物）を取る」です。	00 5. 実コード①：Login.tsx
`from`	`import { 名前 } from "場所";` の一部で、「どこから読み込むか」を書きます。	00 5. 実コード①：Login.tsx
`//`	JavaScript・TypeScript では、`//` から行の終わりまでがコメントです。	00 5. 実コード①：Login.tsx
`const`	JavaScript・TypeScript で、変えない値に名前を付ける書き方。`const db = …;` 。あとで別の値を入れようとするとエラーになります。	00 5. 実コード①：Login.tsx
`=>`	アロー関数（短く書ける関数）の矢印。`(path) => `…${path}`` は「`path` を受け取って文字列を返す関数」。	00 5. 実コード①：Login.tsx
`const [ … ] =`	`const [a, b] = 並び;` は、並びの 0 番目を `a` に、1 番目を `b` に取り出す書き方です（分割代入）。`const [employee_no, setNumber] = useState<string>("");` もこれで、「今の値」と「値を変える関数」の 2 つを受け取っています。	00 5. 実コード①：Login.tsx
`() =>`	`() => { … }` は、名前の無い関数（アロー関数）の書き方です。`()` の中が引数、`=>` の右が中身です。`const Login: React.FC = () => { … }` は「`Login` という部品の関数」、`(e) => setNumber(e.target.value)` のように 1 つの式なら `{ }` を省けます。ほかの関数に「あとで呼んでもらう処理」を渡す時によく使います。	00 5. 実コード①：Login.tsx

コード	意味と使い方	初めて出てくる所
`async`	JavaScript・TypeScript で、中で `await` を使える関数にする印。結果は `Promise`（あとで届く約束）で返ります。	00 5. 実コード①：Login.tsx
`await`	JavaScript・TypeScript で、時間のかかる処理（`Promise`）が終わるまで待って、結果を受け取る書き方。`async` の関数の中で使います。	00 5. 実コード①：Login.tsx
`if`	JavaScript・TypeScript の条件。`if (条件) { … } else { … }`。条件は `( )` で囲みます。	00 5. 実コード①：Login.tsx
`else`	JavaScript・TypeScript で、`if` の条件に合わなかった時の所。`else if (条件)` で、条件をつなげられます。	00 5. 実コード①：Login.tsx
`.post()`	`api.post("/api/login/", データ)` は、axios（サーバーと通信する部品）で、サーバーへデータを送る（POST）メソッドです。`await` を付けて、返事が来るまで待ちます。返事は `response.data` に入ります。	00 5. 実コード①：Login.tsx
`.isAxiosError()`	`axios.isAxiosError(err)` は、起きたエラーが「axios の通信のエラー」かどうかを調べます。`true` なら `err.response?.status`（返事の番号）などが読めます。	00 5. 実コード①：Login.tsx
`?.`	`a?.b` は、`a` が `null` や `undefined` なら、そこで止めて `undefined` にする書き方（オプショナルチェーン）。無いかもしれない物を、エラーにせずにたどれます。	00 5. 実コード①：Login.tsx
`||`	`a || b` は、「a か b のどちらか」（または）です。`el.innerText || el.getAttribute("name") || ""` のように、「左が空なら右を使う」という使い方もします（`0` や `""` も「空」とみなされるので、`null`・`undefined` だけを見たい時は `??` を使います）。	00 5. 実コード①：Login.tsx
`async () =>`	`async () => { … }` は、中で `await`（終わるまで待つ）を使える、名前の無い関数（アロー関数）です。	00 5. 実コード①：Login.tsx
`Number`	`Number("42")` は、文字を数にします。数にできない時は `NaN`（数ではない）になります。	00 5. 実コード①：Login.tsx
`return`	JavaScript・TypeScript で、関数の結果を返して、関数を終える命令。	00 5. 実コード①：Login.tsx
`as`	TypeScript で「この値は、この型として扱う」と伝える書き方（`値 as 型`）。	00 5. 実コード①：Login.tsx
`default`	`export default` は、そのファイルから「いちばんの物」を 1 つ外に出す書き方。読み込む側は `import 名前 from "./ファイル"` と、好きな名前で受け取れます。	00 5. 実コード①：Login.tsx

コード	意味と使い方	初めて出てくる所
`.blur()`	入力欄から入力の場所（フォーカス）を外すメソッド。Enter キーを押した時にキーボードを閉じたり、入力を確定させたりするのに使います。	00 5. 実コード①：Login.tsx
`/* */`	JavaScript・TypeScript では、`/*` から `*/` までがコメントです。何行にもまたがって書けます。	00 5. 実コード①：Login.tsx
`!==`	`a !== b` は、「a と b が同じではない」（型も含めて比べる）です。`===`（同じ）の反対です。	00 5. 実コード①：Login.tsx
`&&`	`a && b` は、「a と b の両方が当てはまる」（かつ）です。左が当てはまらなければ、右は調べません。	00 5. 実コード①：Login.tsx
`条件 ? A : B`	`条件 ? A : B` は、「条件が合えば A、合わなければ B」になる式です（三項演算子）。`user.isAdmin ? "管理者" : "一般"`。`if` とちがい、値を返すので、そのまま変数や表示に使えます。	00 5. 実コード①：Login.tsx
`.target`	イベント（`e`）の `.target` は、イベントが起きたタグ（入力欄など）です。`e.target.value` で、入力欄に今入っている文字を読めます。	00 5. 実コード①：Login.tsx
`.pathname`	URL の `.pathname` は、`/kosu-menu` のような、場所の部分です（`?` から後ろを除きます）。`location.pathname` で、今の画面の URL の場所が分かります。	00 10. 実コード⑥：index.tsx
`let`	JavaScript・TypeScript で、あとで変える値に名前を付ける書き方。	04 2. 変数 — 値に名前を付ける
`%`	`a % b` は、a を b で割った余りです。	04 4. 演算子 — 計算と比較
`!`	前に付けた `!` は、「〜ではない」です。`!isLoading` は「読み込み中ではない」、`if (!data)` は「データが無ければ」。	04 4. 演算子 — 計算と比較
`${…}`	`` `…` ``（バッククォートで囲んだ文字。テンプレート文字列）の中の `${ }` には、値や式を入れられます。`` `エラー ${res.status}` `` は、`res.status` が 404 なら `エラー 404` になります。改行も、そのまま書けます。	04 5. テンプレートリテラル — 文字列に値を埋め込む
`.log()`	`console.log(…)` は、渡した値を画面（ターミナルやブラウザーの開発者ツール）に表示します。	04 6. 条件分岐 — if / else / 三項演算子 /...
`for`	くり返しの書き方。`for (const x of 配列) { … }` は、配列の中身を 1 つずつ `x` に入れてくり返します。	04 7. 繰り返し — for とループ
`++`	`i++` は、`i` を 1 増やします（`i = i + 1` と同じ）。`for (let i = 0; i < 5; i++)` のように、くり返しの数を数えるのによく使います。	04 7. 繰り返し — for とループ

コード	意味と使い方	初めて出てくる所
`.length`	`.length` は、並びの数や、文字の数です。文字の場合は、UTF-16 という数え方なので、絵文字などは 2 と数えることがあります（`[...文字].length` なら、見た目の文字の数に近くなります）。	04 10. 配列 — 値を並べて持つ
`.push()`	JavaScript・TypeScript で、配列の後ろに足すメソッド（元の配列が変わる）。	04 10. 配列 — 値を並べて持つ
`.map()`	配列の中身を 1 つずつ変換した、新しい配列を作るメソッド。	04 11. 配列の必殺技 — map と filter
`.filter()`	配列から、条件に合う物だけを集めた新しい配列を作るメソッド。	04 11. 配列の必殺技 — map と filter
`const { … } =`	`const { name, email } = user;` は、オブジェクトの中の `name` と `email` を、同じ名前の変数に取り出す書き方です（分割代入）。	04 13. 分割代入・スプレッド・オプショナルチェイニング
`...`	スプレッド構文（中身を広げる書き方）。`[...arr1, 3, 4]` は配列の中身を広げて新しい配列を作り、`{ ...base, filter: true }` は、オブジェクトを写して一部だけ足したり変えたりします。	04 13. 分割代入・スプレッド・オプショナルチェイニング
`.create()`	`axios.create({ baseURL: …, withCredentials: true })` は、設定を先に決めた、このアプリ専用の通信の係を作ります。作った係（`api`）で `api.get(…)`・`api.post(…)` と呼ぶと、毎回同じ設定で通信できます。	04 17. 実コード全解説：axios.ts 全8行
`new`	クラスから、新しい物を作る書き方。`new Date(dateStr)`（日付の文字から日付の物を作る）・`new Error("…")` のように使います。	04 19. 実コード全解説：KosuList.tsx の型と関…
`.getDay()`	日付（`Date`）の曜日を、数で返します。0 が日曜日、1 が月曜日、…、6 が土曜日です。`["日","月",…][date.getDay()]` で曜日の字になります。	04 19. 実コード全解説：KosuList.tsx の型と関…
`Date`	`new Date(文字)` は、日時の値を作ります。`.toLocaleString("ja-JP")` で、日本の形の日時の文字になります。	04 19. 実コード全解説：KosuList.tsx の型と関…

コード	意味と使い方	初めて出てくる所
`.get()`	`api.get("/api/kosu_list/", { params: { … } })` は、axios（サーバーと通信する部品）で、サーバーからデータを取ってくる（GET）メソッドです。`params` に書いた物は、URL の `?page=1&…` になって送られます。`await` を付けて返事を待ち、中身は `response.data` に入ります。	04 19. 実コード全解説：KosuList.tsx の型と関…
`Math`	`Math` は、計算の関数が集まった物です。`Math.round(3.6)` は四捨五入で `4`、`Math.max(1, 5)` は大きい方の `5` です。	04 19. 実コード全解説：KosuList.tsx の型と関…
`.createElement()`	`document.createElement("tr")` は、新しい HTML のタグ（要素）を作る、ブラウザーの関数です。React を使わずに画面を作ると、このように 1 つずつ作って入れる手間がかかります。	05 0. 前提知識：Reactが解決した問題
`.appendChild()`	作ったタグを、ページの中の決めた所（親）の最後に入れるメソッド。`tr.appendChild(td)` は、行（`tr`）の中に、ます（`td`）を入れます。	05 0. 前提知識：Reactが解決した問題
`.textContent`	`.textContent` は、タグの中の文字です。入れた文字は、HTML としてではなく、ただの文字として表示されるので、安全です（`innerHTML` に入れると、タグとして動いてしまいます）。	05 0. 前提知識：Reactが解決した問題
`({ 名前 })`	関数の引数の `({ status, label })` は、渡されたオブジェクトから、`status` と `label` を取り出して受け取る書き方です。React の部品（コンポーネント）が props を受け取る時によく使います。	05 3. props ― 親から子へ渡す設定値
`.value`	`.value` は、入力欄に今入っている文字や、クッキーなどの値です。	05 8. 実コード全解説：KosuList.tsx 全215行
`/…/`	JavaScript の `/…/` は、正規表現（文字の形を表す書き方）をそのまま書く形です。`/\d+/` は「数字が 1 つ以上」。`.match(/…/)`・`.test(…)`・`.split(/…/)` などで使います。記号ごとの意味は、この一覧の「正規表現」の表にあります。	05 8. 実コード全解説：KosuList.tsx 全215行
`/…/g`	正規表現の後ろの `g` は、「合う所を全部」という指定（フラグ）です。`.replace(/a/g, "b")` は、全部の `a` を `b` にします（`g` が無いと、最初の 1 つだけ）。`i` は大文字・小文字を区別しない指定です。	05 8. 実コード全解説：KosuList.tsx 全215行

コード	意味と使い方	初めて出てくる所
`Error`	`new Error("説明")` は、エラーを作ります。`throw new Error(…)` でエラーを起こし、`try { … } catch (err) { … }` で受け止めます。`err.message` で説明の文を読めます。	05 8. 実コード全解説：KosuList.tsx 全215行
`setTimeout()`	`setTimeout(() => { … }, 3000)` は、3000 ミリ秒（3 秒）たってから、中の処理を 1 回動かします。返ってくる番号を `clearTimeout(番号)` に渡すと、取りやめにできます。	05 10. 実コード全解説：Loading.tsx 全42行
`clearTimeout()`	`clearTimeout(timer)` は、`setTimeout` で予約した処理を取りやめにします。React では、部品が消える時の後片づけ（`useEffect` の `return () => …`）で呼びます。	05 10. 実コード全解説：Loading.tsx 全42行
`defineConfig()`	`defineConfig({ … })` は、Vite・Vitest の設定ファイル（このアプリでは `frontend/vite.config.ts`）で、設定を書くための関数です。書き間違いを、エディターが教えてくれるようになります。	12 6. フロントのテスト設定ファイル

TypeScript

コード	意味と使い方	初めて出てくる所
`<string>`	`useState<string>("")` の `<string>` は「この入れ物には文字列（string）を入れる」と型を指定する書き方（ジェネリクス）です。HTML のタグではありません。	00 5. 実コード①：Login.tsx
`名前<型>()`	`useState<string>("")` や `useRef<HTMLDivElement>(null)` のように、関数の名前の後ろの `< >` は、「どの型で扱うか」を指定する書き方です（ジェネリクス）。	00 5. 実コード①：Login.tsx
`string`	`string` は、文字の型です。	00 5. 実コード①：Login.tsx
`try`	失敗するかもしれない処理を `try { … }` で囲み、失敗した時の処理を `catch` に書く書き方。	00 5. 実コード①：Login.tsx
`catch`	`try` の中で失敗した時に動く所。`catch (e)` の `e` にエラーが入ります（何が来るか分からないので、使う前に確かめます）。	00 5. 実コード①：Login.tsx
`===`	「型も値も同じか」を比べる書き方。`==` は型を勝手にそろえてから比べて思わぬ結果になるので、ふつうは `===` を使います（違うかは `!==`）。	00 5. 実コード①：Login.tsx

コード	意味と使い方	初めて出てくる所
`instanceof`	値が、そのクラスから作られた物かを調べる書き方。`body instanceof FormData` は「ファイル付きのフォームの中身か」。	00 5. 実コード①：Login.tsx
`type`	`type` は、TypeScript で型に名前を付ける書き方です（`type Props = { isLoading: boolean };` のように、オブジェクトの形などを決めます）。なお、`e.target.type` のように `.` の後ろに来る `type` は、入力欄の種類（`"text"`・`"textarea"` など）を表す、ただの項目の名前です。	00 5. 実コード①：Login.tsx
`export`	「このファイルで作った物を、ほかのファイルから使えるようにする」印。`export default Login;` は「このファイルの代表として `Login` を出す」という意味で、ほかのファイルでは `import Login from './MainPage/Login';` と取り込みます。	00 5. 実コード①：Login.tsx
`HTMLInputElement`	入力欄（`<input>`）を表す型。`e.target instanceof HTMLInputElement` は「イベントが起きた場所が入力欄かどうか」を調べ、`as HTMLInputElement` は「入力欄として扱う」と TypeScript に伝えます。	00 5. 実コード①：Login.tsx
`[キー: string]: 型`	`{ [key: string]: string }` は「キー（名前）が何でもよい文字列で、値も文字列」の、辞書のようなオブジェクトの型です。`routeTitles["/login"]` のように、好きなキーで値を引けます。	00 10. 実コード⑥：index.tsx
`.getElementById()`	`document.getElementById("root")` は、`id="root"` のタグを探して返す関数。React は、ここに画面を描きます。最後の `!` は「必ずある」と TypeScript に伝える印です。	00 10. 実コード⑥：index.tsx
`HTMLElement`	`HTMLElement` は、ページの中のタグ（要素）1 つの型です。`as HTMLElement` で、「これはタグです」と TypeScript に伝えます。	00 10. 実コード⑥：index.tsx
`null`	「値が無い」ことを、わざと表す値。「できない操作なら `null` を返す」のように使います。	04 3. データ型 — 値の種類
`typeof`	値の種類を文字で返す書き方。`typeof value === "number"` は「数か」を調べます。調べたあとの `if` の中では、TypeScript がその型として扱ってくれます。	04 3. データ型 — 値の種類
`switch`	値によって処理を分ける書き方。`switch (値) { case "1": … break; case "2": … break; default: … }`。合う `case` から下を動かすので、それぞれの終わりに `return` か `break` を書きます。	04 6. 条件分岐 — if / else / 三項演算子 /…
`case`	`switch` の中の「この値の時は」の部分。	04 6. 条件分岐 — if / else / 三項演算子 /…

コード	意味と使い方	初めて出てくる所
`of`	`for (const 名前 of 配列)` の一部で、「配列の中身を 1 つずつ」という意味です（ `in` を使うと、中身ではなく番号が出てくるので注意）。	04 7. 繰り返し — for とループ
`function`	関数（ひとまとまりの処理）を作る書き方。`function greet(name: string): string { … }` は、「文字列の `name` を受け取って、文字列を返す関数 `greet` 」です。	04 8. 関数 — 処理に名前を付けて再利用
`number`	数を表す型。整数も小数も `number` です。`let age: number = 42;` 。	04 14. TypeScript — 型という安全装置
`boolean`	`boolean` は、`true` （はい）か `false` （いいえ）の型です。	04 14. TypeScript — 型という安全装置
`A | B`	`string | number` の `|` は「このどれか」を表す型です（ユニオン型）。`value: string | number` は「文字でも数でもよい」。`string | null` は「文字か、まだ無い（null）」で、特によく使います。	04 14. TypeScript — 型という安全装置
`interface`	オブジェクトの形に名前を付ける書き方（ `type` とほぼ同じ）。`interface Employee { employeeNo: number; name: string; }` は「番号（数）と名前（文字）を持つ物」です。	04 14. TypeScript — 型という安全装置
`finally`	`try … catch` のあと、成功しても失敗しても最後に必ず動く所。片づけ（サーバーを閉じる など）を書きます。	04 15. 非同期処理 — Promise と async /…
`.ceil()`	`Math.ceil(x)` は、小数を切り上げて整数にする関数です。`Math.ceil(response.data.count / pageSize)` は「全部の件数を 1 ページの件数で割って、切り上げ」＝ページの数です（45 件を 20 件ずつなら 3 ページ）。	04 19. 実コード全解説：KosuList.tsx の型と関…
`void`	「何も返さない」ことを表す型。`Promise<void>` は「終わったことだけを知らせる約束」です。	05 3. props — 親から子へ渡す設定値
`名前?:`	`buttonColor?: string;` のように名前の後ろに `?` を付けると、「この項目は、無くてもよい」という意味になります。無い時の値は `undefined` です。	05 3. props — 親から子へ渡す設定値
`型[]`	`string[]` のように型の後ろに `[]` を付けると、「その型の並び（配列）」という型になります。`Array<string>` と書いても同じです。	05 4. state と useState — 画面が覚える値

コード	意味と使い方	初めて出てくる所
`.addEventListener()`	出来事（画面の大きさが変わった・キーが押された など）が起きた時に動く関数を登録するメソッド。`window.addEventListener("resize", updateDimensions)` は、窓の大きさが変わるたびに `updateDimensions` を動かします。	05 5. useEffect ― タイミングで処理を走らせる
`.removeEventListener()`	`addEventListener` で登録した関数を、外すメソッド。`useEffect` の片づけで呼んで、同じ関数が何度も登録されるのを防ぎます。	05 5. useEffect ― タイミングで処理を走らせる
`<HTMLDivElement>`	`useRef<HTMLDivElement>(null)` の `<HTMLDivElement>` は「この ref は `<div>` の部品を指す」と型を指定する書き方です。HTML のタグではありません。	05 6. useRef ― 再描画を起こさない箱
`<HTMLHeadingElement>`	`useRef<HTMLHeadingElement>(null)` は「この ref は見出し（`<h1>` 〜 `<h6>`）を指す」という型の指定です。HTML のタグではありません。	05 6. useRef ― 再描画を起こさない箱
`<HTMLElement>`	`React.RefObject<HTMLElement | null>` の `<HTMLElement>` は「画面の部品（どのタグでもよい）」を表す型の指定です。HTML のタグではありません。	05 11. 実コード全解説：TableContainer.ts…
`any`	「何の型でもよい」という特別な型。型の確かめが効かなくなるので、ふだんは使わず、テストでにせ物を作る時など、どうしても必要な所だけに使います。	12 4. 実コード全解説：Login.test.tsx 全13…
`.forEach()`	配列の中身を 1 つずつ処理するメソッド（新しい配列は作らない）。`(x, i) => …` で、番号 `i` も受け取れます。	15 8. テスト

React

コード	意味と使い方	初めて出てくる所
`React`	React の本体。`import React, { useState } from "react";` の `React` は、`React.FC`（部品の型）や `React.useEffect` のように、React の道具を `React.〜` の形で使うための名前です。	00 5. 実コード①：Login.tsx
`Link`	React Router（画面の切り替えの部品）の、リンクの部品。`<Link to="/help">` は `<a href="/help">` に似ていますが、ページ全体を読み直さずに、画面だけを切り替えます。	00 5. 実コード①：Login.tsx

コード	意味と使い方	初めて出てくる所
`useNavigate()`	React Router の道具。`const navigate = useNavigate();` で「画面を切り替える関数」を受け取り、`navigate("/")` のように、プログラムの中から好きな画面へ移ります（ログインに成功したらメニューへ、など）。	00 5. 実コード①：Login.tsx
`.preventDefault()`	ブラウザーがふつうにする動き（フォームを送るとページを移動するなど）を止めるメソッド。React でフォームを扱う時は、`onSubmit` の最初で `e.preventDefault()` を呼びます。	00 5. 実コード①：Login.tsx
`<div>`	特別な意味の無い箱のタグ（JSX の中でも HTML と同じ）。JSX では `class` ではなく `className` で CSS の class を付けます。	00 5. 実コード①：Login.tsx
`<img>`	JSX（React の画面の書き方）での画像のタグ。`<img src={logo} alt="…" />` のように、`src` に `{ }` で画像を入れ、最後を `/>` で閉じます。	00 5. 実コード①：Login.tsx
`<h2>`	2 番目の大きさの見出しのタグ。	00 5. 実コード①：Login.tsx
`<button>`	ボタンのタグ。`onClick={() => …}` で、押した時にすることを決めます。	00 5. 実コード①：Login.tsx
`<Link>`	`<Link to="行き先のURL">…</Link>` と書いて、押すとその画面へ切り替わるリンクを作ります。`to=` が行き先です。	00 5. 実コード①：Login.tsx
`{/* */}`	JSX の中のコメントは、`{/* … */}` と書きます。	00 5. 実コード①：Login.tsx
`{条件 && …}`	JSX の `{errorMessage && (<div role="alert">…</div>)}` は、「条件が合う時（ここではエラーの文がある時）だけ、右を表示する」書き方です。合わない時は、何も表示しません。	00 5. 実コード①：Login.tsx
`className=`	JSX では、HTML の `class` の代わりに `className` と書きます（`class` は JavaScript の言葉として使われているからです）。`className="btn-primary"`。	00 5. 実コード①：Login.tsx
`src=`	`<img src={logo} />` の `src` は、表示する画像。React では `import logo from "../img/….png";` で取り込んだ画像を `{ }` で渡します。	00 5. 実コード①：Login.tsx
`alt=`	`<img alt="業務工数システムロゴ" />` の `alt` は、画像の説明の文。画像が出ない時や、読み上げソフトが読む時に使われます。	00 5. 実コード①：Login.tsx

コード	意味と使い方	初めて出てくる所
`onSubmit=`	`<form onSubmit={handleSubmit}>` は、フォームを送った時（送信ボタン・Enter）に動く処理です。中で `e.preventDefault()` を呼ぶと、ページの読み直しを止められます。	00 5. 実コード①：Login.tsx
`onKeyDown=`	キーが押された時に動かす関数を渡す書き方。`onKeyDown={(e) => { if (e.key === "Enter") … }}` で「Enter キーが押されたら…」を書けます。	00 5. 実コード①：Login.tsx
`htmlFor=`	JSX では、`<label>` の `for` の代わりに `htmlFor` と書きます。	00 5. 実コード①：Login.tsx
`type=`	`<input type="date">`・`<button type="submit">` の `type` は、入力欄やボタンの種類です。	00 5. 実コード①：Login.tsx
`id=`	`id` は、ページの中で 1 つだけの名前です。`<label htmlFor="title">` と組で使います。	00 5. 実コード①：Login.tsx
`value=`	入力欄の `value={employee_no}` は、「入力欄に表示する値は、この変数」という結び付けです。`onChange` で変数を変えると、表示も変わります（制御されたコンポーネント）。	00 5. 実コード①：Login.tsx
`onChange=`	`onChange={(e) => { … }}` は、入力欄の中身が変わるたびに動く処理です。`e.target.value` が、今入っている文字です。	00 5. 実コード①：Login.tsx
`role=`	`role="alert"` は、その部分の役わりを、読み上げソフトに伝える印です。`alert` は、すぐ知らせる大事なお知らせです。	00 5. 実コード①：Login.tsx
`ReactDOM`	React で作った画面を、ブラウザーのページ（HTML）に描く係。`import ReactDOM from 'react-dom/client';` で取り込みます。	00 10. 実コード⑥：index.tsx
`BrowserRouter`	React Router の土台の部品。ブラウザーの URL と画面の切り替えを結び付けます。アプリ全体をこの部品で囲みます。`BrowserRouter as Router` は「`Router` という短い名前で使う」という意味です。	00 10. 実コード⑥：index.tsx
`Router`	ここでは `BrowserRouter` に付けた短い名前（`import { BrowserRouter as Router }`）。`<Router>…</Router>` でアプリ全体を囲み、URL による画面の切り替えができるようにします。	00 10. 実コード⑥：index.tsx
`Routes`	React Router の部品。中に並べた `<Route>` の中から、今の URL に合う 1 つを選んで表示します。	00 10. 実コード⑥：index.tsx
`Route`	React Router の部品。「この URL の時は、この画面」という 1 つの組み合わせを表します。	00 10. 実コード⑥：index.tsx

コード	意味と使い方	初めて出てくる所
`.useEffect()`	`React.useEffect(() => {…}, [location])` は、`useEffect` を `React.` を付けて呼ぶ書き方です。画面が出た後や、`[ ]` の中の値が変わった後に、中の処理を動かします。	00 10. 実コード⑥：index.tsx
`useLocation()`	React Router の道具。`const location = useLocation();` で「今の画面の URL の情報」を受け取ります。`location.pathname` が `/login` のような URL の道の部分です。	00 10. 実コード⑥：index.tsx
`useEffect()`	「画面を描いたあとに、することがある」時に使う関数（フック）。`useEffect(() => { … }, [見張る値])` で、データを取ってくる・イベントを登録する、などをします。`[ ]` の中の値が変わった時だけ、もう一度動きます。	00 10. 実コード⑥：index.tsx
`<Routes>`	`<Routes>` の中に `<Route path="…" element={…} />` を並べます。URL に合う `<Route>` の画面だけが表示されます。	00 10. 実コード⑥：index.tsx
`<Route>`	`<Route path="/kosu-new" element={<KosuNew />} />` は「URL が `/kosu-new` の時は `KosuNew` の画面を出す」という意味です。`path="/kosu-update/:id"` の `:id` は「ここには何が来てもよい（番号など）」という印で、画面の中で `useParams()` で読めます。	00 10. 実コード⑥：index.tsx
`.createRoot()`	`ReactDOM.createRoot(document.getElementById('root'))` は、HTML の `<div id="root">` を React の画面の置き場所にします。その後 `root.render(<App />)` で画面を描きます。	00 10. 実コード⑥：index.tsx
`.render()`	`createRoot(要素).render(<App />)` は、React の部品（ここでは `<App />`）を、その要素の中に表示します。React の画面は、ここから始まります。	00 10. 実コード⑥：index.tsx
`<Router>`	`<Router>…</Router>`（ここでは `BrowserRouter` のこと）でアプリ全体を囲むと、中で `<Routes>` や `useNavigate` が使えるようになります。	00 10. 実コード⑥：index.tsx
`onClick=`	`onClick={() => …}` は、押された時に動く処理です。	04 9. アロー関数 — 関数の短い書き方
`useCallback()`	関数を覚えておき、描き直しのたびに作り直さないようにするフック。`useEffect` の見張る値に関数を入れる時などに使います。	04 19. 実コード全解説：KosuList.tsx の型と関…
`<p>`	段落のタグ（JSX の中でも HTML と同じ）。`return <p>読み込み中…</p>;` のように、部品が返す画面に使います。	04 19. 実コード全解説：KosuList.tsx の型と関…

コード	意味と使い方	初めて出てくる所
`<TableContainer>`	このアプリの表の枠の部品（`Components/TableContainer.tsx`）。`<TableContainer …>` の中に表を入れると、決まった見た目で出ます。	04 19. 実コード全解説：KosuList.tsx の型と関…
`<table>`	表のタグ（JSX の中でも HTML と同じ）。	04 19. 実コード全解説：KosuList.tsx の型と関…
`<thead>`	表の見出しの行をまとめるタグ。	04 19. 実コード全解説：KosuList.tsx の型と関…
`<tbody>`	表の中身の行をまとめるタグ。	04 19. 実コード全解説：KosuList.tsx の型と関…
`<tr>`	表の 1 行のタグ。	04 19. 実コード全解説：KosuList.tsx の型と関…
`<td>`	表の中身のますのタグ。`<td>{r.id}</td>` のように、`{ }` の中に値を書けます。	04 19. 実コード全解説：KosuList.tsx の型と関…
`<Pagination>`	このアプリのページ送りの部品（`Components/Pagination.tsx`）。一覧が長い時に「前へ・次へ・ページの番号」を出します。1 ページの件数は、管理者設定（`administrator_data` の `menu_row`）で決まります（設定が無い時は 20 件）。	04 19. 実コード全解説：KosuList.tsx の型と関…
`key=`	`key={r.id}` は、並びを `map` で表示する時に、1 つずつに付ける「見分けの印」です。React は、これで、どれが増えた・消えた・動いたかを見分けます。並びの中で重ならない値（番号など）にします。	04 19. 実コード全解説：KosuList.tsx の型と関…
`<>…</>`	JSX の `<> … </>` は、名前の無い入れ物（フラグメント）です。部品は 1 つの物を返す決まりなので、いくつも並べたい時に、余分なタグを増やさずに包みます。	05 2. JSX ― JavaScriptの中のHTML
`.current`	`useRef` で作った入れ物の中身。`searchDayRef.current = "";` のように読み書きします。書き換えても画面は描き直されません（`useState` との違い）。	05 5. useEffect ― タイミングで処理を走らせる
`ref=`	`<div ref={searchBarRef}>` は、`useRef` で作った入れ物に「この `<div>` の部品」を結び付けます。あとで `searchBarRef.current` から、その部品の大きさなどを読めます。	05 6. useRef ― 再描画を起こさない箱

コード	意味と使い方	初めて出てくる所
`TableContainer`	このアプリの部品（`frontend/src/Components/TableContainer.tsx`）。一覧の表を、決まった見た目と、ページ送り付きで出す枠です。	05 8. 実コード全解説：KosuList.tsx 全215行
`Loading`	このアプリの「読み込み中」の部品（`Components/Loading.tsx`）。サーバーの返事を待つ間に出します。	05 8. 実コード全解説：KosuList.tsx 全215行
`<Loading>`	`<Loading isLoading={loading} />` は、`loading` が `true` の間、「読み込み中」の印を出すこのアプリの部品です。	05 8. 実コード全解説：KosuList.tsx 全215行
`<h1>`	いちばん大きな見出しのタグ（JSX の中でも HTML と同じ）。	05 8. 実コード全解説：KosuList.tsx 全215行
`<nav>`	メニューを表すタグ。	05 8. 実コード全解説：KosuList.tsx 全215行
`defaultValue=`	入力欄の「最初に入っている値」。`value=` と違い、その後は利用者が自由に書き換えられます（React が値を見張らない入力欄になります）。	05 8. 実コード全解説：KosuList.tsx 全215行
`placeholder=`	`placeholder="日付を選択"` は、入力欄が空の時に、うすく見せる案内の文字です。	05 8. 実コード全解説：KosuList.tsx 全215行
`<th>`	表の見出しのますのタグ。	05 8. 実コード全解説：KosuList.tsx 全215行
`<span>`	文の一部分を囲む、特別な意味の無いタグ（JSX の中でも HTML と同じ）。状態の札など、一部だけ見た目を変える時に使います。	05 9. 実コード全解説：Pagination.tsx 全71行
`style={{ … }}`	JSX で部品に直接 CSS を書く書き方。外側の `{ }` は「ここから JavaScript」、内側の `{ }` は「CSS の項目をまとめたオブジェクト」です。`style={{ color: "red", marginTop: 8 }}` のように、CSS の名前は `margin-top` ではなく `marginTop`（つなぎの `-` を取って次を大文字）と書きます。	05 9. 実コード全解説：Pagination.tsx 全71行
`style=`	部品に直接 CSS を当てる指定。値は `style={{ color: "red" }}` のように、CSS の項目をまとめたオブジェクトで渡します。	05 9. 実コード全解説：Pagination.tsx 全71行

コード	意味と使い方	初めて出てくる所
`disabled=`	`disabled={currentPage === 1}` は、条件が true の間、ボタンを押せなくします（1 ページ目では「前へ」を押せない、など）。	05 9. 実コード全解説：Pagination.tsx 全71行
`useState()`	React の部品に「覚えておく値」（state）を作る関数（フック）。`const [employee_no, setNumber] = useState<string>("");` で、今の値 `employee_no` と、値を変える関数 `setNumber` ができます。`setNumber(新しい値)` を呼ぶと、画面が描き直されます。	05 10. 実コード全解説：Loading.tsx 全42行
`ReactNode`	「画面に出せる物なら何でも」を表す型。文字・数・JSX の部品・それらの並びなどが入ります。部品の `children`（中身）の型によく使います。	05 11. 実コード全解説：TableContainer.ts...
`react()`	`react()` は、Vite に、React（JSX）のファイルを扱えるようにする部品（プラグイン）です。	12 6. フロントのテスト設定ファイル
`name=`	部品（`<input>` など）の名前。フォームを送る時の項目の名前や、テストで部品を探す目印に使います。`id=` と同じ名前にしておくことが多いです。	15 6. 不具合
`<WorkSelect>`	就業形態を選ぶプルダウンの部品（`frontend/src/Components/WorkSelect.tsx`）。`mode` を省くと出勤を伴う物だけ、`mode="ALL"` で休日なども含めた全部の選択肢を出します。	15 8. テスト

Python

コード	意味と使い方	初めて出てくる所
`from`	`from パッケージ import 名前` の一部で、「どこから読み込むか」を書きます。`from pathlib import Path`。	00 6. 実コード②：kosu/urls.py
`import`	ほかのモジュール（部品）を読み込む書き方。`import re` で正規表現の道具、`from pathlib import Path` で `Path` だけを使えるようにします。	00 6. 実コード②：kosu/urls.py
`#`	Python では、`#` から行の終わりまでがコメントです。	00 6. 実コード②：kosu/urls.py
`名前=値`	`open(path, encoding="utf-8")` の `encoding="utf-8"` のように、`名前=値` で渡す引数は「キーワード引数」です。順番に関係なく、名前で渡せます。	00 6. 実コード②：kosu/urls.py

コード	意味と使い方	初めて出てくる所
`+=`	`total += n` は `total = total + n` の短い形です。	00 6. 実コード②：kosu/urls.py
`Path`	`Path`（`pathlib`）は、ファイルやフォルダの場所を表す型です。`Path(__file__).resolve().parent.parent` は「このファイルの場所から 2 つ上のフォルダ」＝プロジェクトのいちばん上のフォルダです。`/` でつなげます（`BASE_DIR / "media"`）。	00 7. 実コード③：main_views.py の Login
`class`	クラス（データと、それを扱う関数をまとめた設計図）を作る書き方。`class member(models.Model):` の次から、字下げした所が中身です。`( )` の中は、元にするクラス（ここでは Django のモデル）です。	00 7. 実コード③：main_views.py の Login
`def`	関数を作る書き方。`def greet(name: str) -> str:` の次の行から、**字下げした所が関数の中身**です。Python は `{ }` の代わりに字下げで「どこまでが中身か」を表すので、字下げをそろえることがとても大事です。	00 7. 実コード③：main_views.py の Login
`try`	Python で、失敗するかもしれない処理を `try:` の中に書き、失敗した時の処理を `except:` に書く書き方。	00 7. 実コード③：main_views.py の Login
`except`	`try:` の中でエラーが起きた時に動く所。`except json.JSONDecodeError:` は「JSON として読めなかった時」、`except Exception as e:` は、ほとんど全部のエラーを `e` という名前で受け取ります。	00 7. 実コード③：main_views.py の Login
`return`	Python の関数の結果を返して、関数を終える命令。	00 7. 実コード③：main_views.py の Login
`if`	Python の条件。`if 条件:` の次の行から、字下げした所が中身です。`a if 条件 else b` の形で、1 行の中で値を選ぶこともできます。	00 7. 実コード③：main_views.py の Login
`not`	「〜ではない」。`if not def_Ver:` は「`def_Ver` が無ければ（`None` や空なら）」。	00 7. 実コード③：main_views.py の Login
`else`	`if` の条件に合わなかった時の所。`if … else …` の 1 行の形でも使います。	00 7. 実コード③：main_views.py の Login

コード	意味と使い方	初めて出てくる所
`.loads()`	文字列から読み取るメソッド。`json.loads(request.body)` は、送られてきた JSON の文字を、Python の辞書にします。	00 7. 実コード③：main_views.py の Login
`.get()`	辞書（`dict`）から、キーで値を取り出すメソッド。`data.get('employee_no')` は、キーが無ければ `None`（2 つ目の値を書けば、その値）を返すので、`data['employee_no']` と違ってエラーになりません。	00 7. 実コード③：main_views.py の Login
`self`	クラスの中の関数（メソッド）の最初の引数 `self` は、「その関数を呼んだ、その物自身」です。`self.name` で、自分の項目を読み書きします。呼ぶ時は、`self` は書きません（`obj.save()`）。	00 7. 実コード③：main_views.py の Login
`True`	Python の「正しい（はい）」の値。反対は `False`。頭は大文字です（`true` ではありません）。	00 7. 実コード③：main_views.py の Login
`as`	Python で、別の名前を付ける書き方。`except Exception as error:`（起きたエラーを `error` で受け取る）・`import numpy as np`・`with open(…) as f:` のように使います。	00 7. 実コード③：main_views.py の Login
`Exception`	`Exception` は、Python のふつうのエラー全部の、元になる型です。`except Exception as e:` は、ほとんど全部のエラーを受け止めます。どんなエラーか分からない時の最後の守りに使います。	00 7. 実コード③：main_views.py の Login
`__str__`	`def __str__(self):` は、`str(物)` や `print(物)`、管理画面の一覧で表示する時の文字を決める関数です。	00 8. 実コード④：models.py
`str()`	値を文字列に変える関数。`str(uuid.uuid4())` は、UUID を `"3f2a…"` のような文字にします。数と文字列は `==` で比べても等しくならないので、そろえる時にも使います。	00 8. 実コード④：models.py
`for`	Python のくり返し。`for x in 並び:` の次の行から、字下げした所が中身です。`[式 for x in 並び]` の形（内包表記）で、新しいリストを 1 行で作ることもできます。	00 8. 実コード④：models.py
`in`	`for x in 並び`（並びから 1 つずつ）と、`値 in 並び`（含まれているか）の 2 つの使い方があります。	00 8. 実コード④：models.py
`super()`	親のクラス（元にしたクラス）を呼び出す書き方。`super().save(*args, **kwargs)` は「自分で足した処理の後で、元々の保存の処理も動かす」という意味です。	00 8. 実コード④：models.py

コード	意味と使い方	初めて出てくる所
`**kwargs`	`def save(self, *args, **kwargs):` の `**kwargs` は、「名前付きの引数を、いくつでも受け取り、辞書にまとめる」書き方です。`super().save(*args, **kwargs)` で、受け取った物を、そのまま元の `save` に渡しています。	00 8. 実コード④：models.py
`*args`	`def f(*args):` の `*args` は、「いくつでも引数を受け取り、タプル（並び）にまとめる」書き方です。	00 8. 実コード④：models.py
`__class__`	その物を作ったクラスを指す名前。`self.__class__.objects.count()` は「自分と同じモデルの、表の行の数」を数えます。	00 8. 実コード④：models.py
`r"…"`	`r"…"` は「生の文字列」で、中の `\` を特別な意味にしません。正規表現の `r"\d{6}"` のように、`\` の多い文字をそのまま書けます。	00 9. 実コード⑤：hozen_another/urls.py
`.resolve()`	`Path(__file__).resolve()` は、ファイルの場所を、`C:\` から始まる絶対の場所にするメソッド。`.parent` で 1 つ上のフォルダになります。	00 12. 実コード⑦：settings.py の主要設定を1…
`.join()`	Python で、区切り文字の方に付けて、リストの中身をつなぐメソッド。`" ".join(["a", "b"])` は `"a b"`（TypeScript とは書く順番が逆なので注意）。	00 12. 実コード⑦：settings.py の主要設定を1…
`__file__`	`__file__` は、今のファイルの場所が入った、特別な変数です。`Path(__file__).resolve().parent` で、このファイルが置いてあるフォルダが分かります。	00 12. 実コード⑦：settings.py の主要設定を1…
`RotatingFileHandler`	ログ（記録）をファイルに書く係のうち、ファイルが大きくなったら新しいファイルに切り替える物。`maxBytes`（1 つのファイルの最大の大きさ）と `backupCount`（古いファイルを何個残すか）を決めます。	00 12. 実コード⑦：settings.py の主要設定を1…
`.parent`	`Path` の `.parent` は、1 つ上のフォルダです。`.parent.parent` で 2 つ上です。	00 12. 実コード⑦：settings.py の主要設定を1…
`.getenv()`	`os.getenv('DB_NAME', '')` は、環境変数（パソコンや Azure に設定した値）を読みます。無い時は 2 番目の値（ここでは空の文字）が返ります。	00 12. 実コード⑦：settings.py の主要設定を1…
`False`	Python の「正しくない（いいえ）」の値。	00 12. 実コード⑦：settings.py の主要設定を1…

コード	意味と使い方	初めて出てくる所
`print()`	渡した値を画面に表示する関数。`print("a", 1)` のように、いくつも渡すと空白でつないで表示し、最後に改行します。	06 1. インデント — Python最大の特徴
`.strip()`	文字列の前後の空白や改行を取り除いたものを返すメソッド。	06 1. インデント — Python最大の特徴
`.info()`	ログ（記録）に「お知らせ」の重さで 1 行書きます。重さは `debug` < `info` < `warning` < `error` の順です。	06 1. インデント — Python最大の特徴
`int()`	文字列などを整数にする関数。`int("25")` は 25。数でない文字だと `ValueError` です。	06 2. 変数とデータ型
`.append()`	Python で、リストの後ろに 1 つ足すメソッド。	06 3. リスト・辞書・タプル
`elif`	「そうでなくて、もし〜なら」（`else if` の短い形）。`if … elif … else` で、いくつもの場合に分けます。	06 4. 条件分岐 if
`and`	「かつ」。両方が成り立つ時だけ `True`。左が成り立たない時は、右を調べません。	06 4. 条件分岐 if
`or`	「または」。`a or b` は、`a` が空（`None`・`""`・0 など）なら `b` を使います。`(row.get("番号") or "").strip()` は「無ければ空の文字列にしてから空白を取る」。	06 4. 条件分岐 if
`is`	「同じ物か」を調べる書き方。`if value is None:` は「値が無ければ」。`None` と比べる時は `==` でなく `is` を使います。	06 4. 条件分岐 if
`None`	Python の「値が無い」ことを表す値。調べる時は `x is None` と書きます。	06 4. 条件分岐 if
`.timedelta()`	「時間の長さ」を表す物。`datetime.timedelta(days=1)` は 1 日分です。日付から引くと、その日数前の日付になります。	06 5. 繰り返し for
`range()`	数の並びを作ります。`range(1, 8)` は 1, 2, …, 7（8 は入らない）。`for d in range(1, 8):` で 7 回くり返します。	06 5. 繰り返し for
`[式 for 名前 in 並び]`	`[f"http://{host}" for host in ALLOWED_HOSTS]` は、「並びの 1 つずつから、新しい値を作って、並び（リスト）にする」書き方です（リスト内包表記）。後ろに `if 条件` を付けると、条件に合う物だけになります。	06 5. 繰り返し for
`break`	Python のくり返しを、途中で抜ける命令。	06 5. 繰り返し for
`f"…{値}…"`	`f"…"`（頭に `f` を付けた文字）の中の `{ }` には、変数や式を書けて、その値が入ります（f 文字列）。`f'{member_name} 氏の工数未入力があります。'`。	06 5. 繰り返し for

コード	意味と使い方	初めて出てくる所
`!=`	`a != b` は、「a と b が等しくないか」です。	06 6. リスト内包表記
`is not`	`x is not None` は、「x が None ではない」です。`None` かどうかは、`==` ではなく `is` で調べます。	06 6. リスト内包表記
`with`	使い終わったら自動で片づける書き方。`with path.open(encoding="utf-8") as f:` の中で使ったファイルは、抜ける時に自動で閉じられます。	06 7. 関数 def
`.readlines()`	ファイルの全部の行を、1 行ずつの文字のリストにして読みます。行の終わりの改行も付いたままです。	06 7. 関数 def
`open()`	ファイルを開く関数。`open(パス, encoding="utf-8")` で読む、`open(パス, "w", encoding="utf-8")` で書く形で開きます。`with open(…) as f:` と書くと、使い終わったら自動で閉じます。	06 7. 関数 def
`A if 条件 else B`	`a if 条件 else b` は、「条件が合えば a、合わなければ b」になる式です。`ext = name.rsplit(".", 1)[-1] if "." in name else ""`。	06 8. 三項演算子・f文字列
`==`	`a == b` は、「a と b が等しいか」です。`=`（1 つ）は「値を入れる」です。	06 8. 三項演算子・f文字列
`__init__`	`def __init__(self, name, age):` は、クラスから物を作る時（`Person("山田", 30)`）に、自動で呼ばれる関数です。受け取った値を `self.name = name` のように覚えます。	06 9. クラスとインスタンス
`pass`	「何もしない」という Python の命令。中身が要らないけれど、形の上で何か書く必要がある所に置きます。	06 9. クラスとインスタンス
`ValueError`	`ValueError` は、「型は合っているが、値がおかしい」時のエラーです。`int("abc")` で起きます。	06 10. 例外処理 try-except
`.NamedTemporaryFile()`	名前の付いた一時ファイルを作ります。`suffix=".xlsx"` で名前の終わりを決め、`delete=False` で「閉じても消さない」ようにします（あとで自分で `os.remove` で消します）。	06 11. with 文
`.today()`	`datetime.date.today()` は、今日の日付を返します。	06 12. import と相対 import
`.getLogger()`	名前の付いたログの係を取り出します。`logging.getLogger('views_logger')` の名前は、`settings.py` の `LOGGING` の設定と結び付いています。	06 14. 実コード全解説①：main_views.py の …

コード	意味と使い方	初めて出てくる所
`.stdout`	`sys.stdout` は、`print` の出力先（ふつうは画面）です。このアプリは `sys.stdout = PrintToLogger()` で出力先をすり替え、`print` した文がログ（`web_console.log`）に書かれるようにしています。	06 14. 実コード全解説①：main_views.py の…
`.getmembers()`	`inspect.getmembers(モジュール)` は、モジュールの中にある名前と物の組を全部並べます。モデルのクラスを全部集める時などに使います。	06 15. 実コード全解説②：main_utils.py の検…
`.isclass()`	`inspect.isclass(物)` は、その物がクラスかどうかを調べます。	06 15. 実コード全解説②：main_utils.py の検…
`issubclass()`	`issubclass(A, B)` は「A は B を元にしたクラスか」を調べます。`issubclass(obj, models.Model)` で「Django のモデルか」が分かります。	06 15. 実コード全解説②：main_utils.py の検…
`__module__`	クラスや関数が書かれているモジュール（ファイル）の名前。`obj.__module__ == myapp_models.__name__` で「このアプリの `models.py` で作ったクラスか」を調べます。	06 15. 実コード全解説②：main_utils.py の検…
`__name__`	`__name__` は、今のファイル（モジュール）の名前が入った、特別な変数です。ファイルを直接動かした時は `"__main__"` になります。`type(error).__name__` は、エラーの種類の名前です。	06 15. 実コード全解説②：main_utils.py の検…
`.strftime()`	日時を、自分で決めた形の文字列にするメソッド。`%Y` は年・`%m` は月・`%d` は日・`%H` は時・`%M` は分。	06 16. 実コード全解説③：asynchronous_vie…
`.fromisoformat()`	`date.fromisoformat("2026-12-01")` は、`2026-12-01` の形の文字列から日付を作る関数です。	06 16. 実コード全解説③：asynchronous_vie…
`.uuid4()`	ほかと重ならない、ばらばらな ID（UUID）を作ります。`str(uuid.uuid4())` で `"3f2a…"` のような文字になります。仕事（タスク）の番号などに使います。	06 16. 実コード全解説③：asynchronous_vie…
`local()`	`threading.local()` を短く呼んだ物（`from threading import local`）。スレッド（同時に動く処理）ごとに別々の中身を持つ入れ物を作ります。	06 18. 実コード全解説⑤：signals.py の差分計算
`type()`	値の型を返す関数。`type(error).__name__` は、エラーの種類の名前です。	06 18. 実コード全解説⑤：signals.py の差分計算

コード	意味と使い方	初めて出てくる所
`getattr()`	物から、名前で属性（中の値）を取り出す関数。`getattr(error, "code", 既定値)` は、`code` が無ければ既定値を返します。	06 18. 実コード全解説⑤：signals.py の差分計算
`continue`	くり返しの「今回の残り」を飛ばして、次へ進む命令。	06 18. 実コード全解説⑤：signals.py の差分計算
`isinstance()`	値が、その型かどうかを調べる関数。`isinstance(value, dict)` は「辞書か」。	06 18. 実コード全解説⑤：signals.py の差分計算
`.isoformat()`	日時を `2026-11-02T08:00:00+09:00` のような、決まった形（ISO 8601）の文字列にするメソッド。	06 18. 実コード全解説⑤：signals.py の差分計算
`.update()`	辞書に、別の辞書の中身をまとめて入れる（上書きする）メソッド。Django の問い合わせの `.update(項目=値)` は、合う行をまとめて書きかえる、別の意味です。	07 5. 実コード全解説：member_views.py 全2...
`.local()`	`threading.local()` は、スレッド（同時に動く処理）ごとに別々の中身を持つ入れ物を作ります。同時に来たリクエスト同士で、値が混ざりません。	07 10. 実コード全解説：middleware と apps...
`__call__`	`def __call__(self, request):` を書いたクラスの物は、関数のように `物(request)` と呼べます。Django のミドルウェアは、リクエストのたびにこれが呼ばれます。	07 10. 実コード全解説：middleware と apps...
`.remove()`	`os.remove(ファイルの場所)` は、ファイルを消します。	07 11. 実コード全解説：asynchronous_view...
`.Thread()`	`threading.Thread(target=関数, args=(…))` は、関数を別のスレッド（同時に動く処理）で動かす準備をします。`.start()` で動き出します。重い仕事を裏で動かし、画面をすぐ返すのに使います。	07 11. 実コード全解説：asynchronous_view...
`.start()`	`thread.start()` は、準備したスレッドを動かし始めます。呼んだ側は、終わるのを待たずに先へ進みます。	07 11. 実コード全解説：asynchronous_view...
`len()`	長さ（中身の数）を返す関数。`len("abc")` は 3、`len(detail_list)` はリストの中身の数です。	07 11. 実コード全解説：asynchronous_view...

コード	意味と使い方	初めて出てくる所
`.basename()`	`os.path.basename("C:/a/b/report.xlsx")` は、場所の最後の名前（`report.xlsx`）だけを返します。	07 11. 実コード全解説：asynchronous_view...
`.sleep()`	`time.sleep(3)` は、3 秒待ちます。	07 11. 実コード全解説：asynchronous_view...
`raise`	エラー（例外）を起こして、処理を止める書き方。`raise ValueError("未入力箇所があります")`。呼んだ側は `try … except` で受け止められます。	08 7.【最重要】休憩文字列 #HHMMHHMM の読み方
`.strptime()`	`datetime.strptime("2026/11/02 08:00", "%Y/%m/%d %H:%M")` は、決めた形の文字列から日時を読む関数です。	08 7.【最重要】休憩文字列 #HHMMHHMM の読み方
`.replace()`	`datetime` の `.replace(hour=8, minute=0)` は、一部だけ変えた新しい日時を返すメソッドです（文字列の `.replace("前", "後")` は、置きかえた新しい文字列を返す、別の意味です）。	08 7.【最重要】休憩文字列 #HHMMHHMM の読み方
`.astimezone()`	日時を、別の時間帯（世界標準時・日本時間など）で表したものに変えるメソッド。	08 7.【最重要】休憩文字列 #HHMMHHMM の読み方
`[開始:終わり]`	`text[1:4]` は、1 番目から 4 番目の手前まで（1, 2, 3）を取り出します（スライス）。`[:3]` は最初から、`[3:]` は 3 番目から最後まで、`[::-1]` は逆さまです。	08 7.【最重要】休憩文字列 #HHMMHHMM の読み方
`.index()`	`文字.index(':')` は、`:` が何文字目にあるかを返します（最初は 0）。見つからない時はエラーになります。	08 8. kosu_utils.py 逐行解説①: time_...
`enumerate()`	並びを、番号つきで 1 つずつ取り出す関数。`for i, e in enumerate(detail_list):` は、`i` に 0, 1, 2...、`e` に中身が入ります（`start=1` を付けると 1 から数えます）。	08 13. kosu_utils.py 逐行解説⑥: deta...
`list()`	並び（`keys()` の結果や文字列など）から、新しいリストを作る関数。`list("abc")` は `["a", "b", "c"]`。	08 16. モデル④: kosu_division
`not in`	`val not in [None, '']` は、「`val` が `None` でも空の文字でもない」です。`in` の反対です。	08 16. モデル④: kosu_division
`.makedirs()`	`os.makedirs(フォルダ)` は、フォルダを作ります。途中のフォルダが無ければ、それも作ります。	09 6. tasks.py 全解説①：工数バックアップ

コード	意味と使い方	初めて出てくる所
`.split()`	Python で、文字列を分けてリストにするメソッド。`"a b c".split()` のように何も渡さないと、空白（いくつ続いても）で分けます。	09 7. tasks.py 全解説②：予測データ出力
`max()`	いちばん大きい物を返す関数。`max(3, 9, 4)` は 9、`max([5, 1, 8])` は 8。いちばん小さい物は `min()` です。	09 7. tasks.py 全解説②：予測データ出力
`locals()`	その時の関数の中の変数を、名前と値の辞書で返します。`'temp_file_path' in locals()` で「その変数がもう作られているか」を調べられます。	09 8. tasks.py 全解説③：ロード
`.date()`	日時（`datetime`）から、日付の部分だけを取り出します。	09 9. tasks.py 全解説④：人員・班員・定義・その他…
`.timezone()`	`pytz.timezone('Asia/Tokyo')` は、時間帯（タイムゾーン）を表す物を作ります。日時を日本の時間に直す時に使います。	09 9. tasks.py 全解説④：人員・班員・定義・その他…
`ImportError`	`import` に失敗した時（部品が入っていない時など）のエラー。`except ImportError:` で、無い時の代わりの処理を書けます。	09 9. tasks.py 全解説④：人員・班員・定義・その他…
`.setdefault()`	`os.environ.setdefault('名前', '値')` は「その環境変数がまだ無い時だけ、この値にする」。`DJANGO_SETTINGS_MODULE` で、どの設定ファイルを使うかを決めます。	09 12. wsgi.py 全解説：Webサーバーとの玄関
`"""…"""`	`"""…"""`（`"` を 3 つ）で囲んだ文字は、何行にもまたがって書けます。関数・クラス・ファイルのいちばん最初に書いた物は、その説明書（docstring）になり、`help()` やエディターで読めます。	09 12. wsgi.py 全解説：Webサーバーとの玄関
`.environ`	`os.environ` は、環境変数の入れ物（辞書のような物）です。`os.environ.get("名前", 無い時の値)` で読みます。	09 12. wsgi.py 全解説：Webサーバーとの玄関
`**辞書`	`member.objects.create(…, **break_times)` のように、呼ぶ時に辞書の前に `**` を付けると、辞書の中身を、名前付きの引数としてばらして渡します（`break_time1='#10401130', break_time1_over1='#15101520', …` を全部書いたのと同じ）。	12 8. 実コード全解説：test_backend.py の土台
`.dumps()`	`json.dumps(値, ensure_ascii=False)` は、Python の値を JSON の文字列にする関数です（`ensure_ascii=False` で日本語をそのまま）。	12 8. 実コード全解説：test_backend.py の土台

コード	意味と使い方	初めて出てくる所
`date()`	日付を表す型。`date(2026, 11, 10)` のように作ります。	12 9. 実コード全解説：TestModels
`{値:0幅}`	f 文字列の `{42:05d}` の `05` は、「5 けたになるまで、前を 0 で埋める」指定です（`00042`）。	12 9. 実コード全解説：TestModels
`{値:d}`	f 文字列の `{n:d}` の `d` は、整数として表示する指定です。	12 9. 実コード全解説：TestModels
`__contains__`	`def __contains__(self, item):` を書いたクラスの物は、`item in 物` と書けるようになります。	12 12. バックのテスト設定
`__getitem__`	`def __getitem__(self, item):` を書いたクラスの物は、`物[item]` と書いて中身を取り出せるようになります。	12 12. バックのテスト設定
`TypeError`	値の種類（型）が合わない時のエラー。`int(None)` のように、数にできない種類の値を渡すと出ます。`except (TypeError, ValueError):` と並べると、まとめて捕まえられます。	15 5. 安全の問題
`AttributeError`	その値に無い名前（属性・メソッド）を使った時のエラー。`json.loads()` の結果が辞書ではなくリストだった時に、`data.get(...)` と書くと出ます。	15 5. 安全の問題
`.isdigit()`	文字列が、全部数字の文字だけでできていれば `True` を返すメソッド。`str(a).strip().isdigit()` で、「従業員番号として読める文字か」を確かめています。	15 5. 安全の問題
`set()`	集合（同じ値は 1 つしか入らない入れ物）を作る関数。`seen = set()` で空の集合を作り、`seen.add(x)`・`x in seen` で使います。重複を見つける時に便利です。	15 5. 安全の問題
`setattr()`	`setattr(物, '名前', 値)` で、名前を文字で指定して、その物の属性に値を入れる関数。`setattr(inquir_member, 'pop_up1', '')` は `inquir_member.pop_up1 = ''` と同じです。名前を `f'pop_up{i}'` のように作れるので、`pop_up1` 〜 `pop_up5` のような似た名前の項目を、ループで扱えます。	15 5. 安全の問題
`.is_file()`	`Path` の `.is_file()` は、その場所にファイルがあれば `True` を返すメソッド。フォルダや、何も無い時は `False` です。	15 5. 安全の問題
`.unlink()`	`Path` の `.unlink()` は、そのファイルを消すメソッド。消せない時は `OSError` になります。	15 5. 安全の問題

コード	意味と使い方	初めて出てくる所
`original_close()`	この巻のダウンロードの直し方で、`FileResponse` が元から持っていた `close`（開いたファイルを閉じる処理）を、変数 `original_close` に取っておいた物。先にこれを呼んでファイルを閉じてから、ファイルを消します。	15 5. 安全の問題
`OSError`	ファイルを開く・消すなど、OS とのやり取りで起きるエラーのまとめ役。`except OSError:` で、ファイルが無い・使用中で消せない、などをまとめて受け止めます。	15 5. 安全の問題
`.compare_digest()`	`hmac.compare_digest(a, b)` は、2 つの値が同じかを、かかる時間から中身を推測されない方法で比べる関数です。合言葉やハッシュを比べる時に使います。	15 5. 安全の問題
`.encode()`	文字列を、バイトの並びにするメソッド（UTF-8）。`token.encode()` のように、ハッシュの計算など、バイトを求める関数に渡す前に使います。	15 5. 安全の問題
`.wraps()`	`@functools.wraps(func)` は、デコレーターの中で作った関数に、元の関数（`func`）の名前や説明を引き継がせる印。付けないと、エラーの表示などで、元の関数の名前が分からなくなります。	15 6. 不具合
`func()`	デコレーター `atomic_load` に渡された、元の関数（ロード関数）を呼んでいる所。`func(*args, **kwargs)` の `*args`・`**kwargs` は、「受け取った引数を、全部そのまま渡す」書き方です。	15 6. 不具合
`@functools`	`@functools.wraps(func)` の頭の部分。`functools`（関数を扱う道具を集めた、Python に最初から入っている部品）の `wraps` を、デコレーターとして使うという意味です。	15 6. 不具合
`@atomic_load`	この巻で `kosu/tasks.py` に足す、自分で作ったデコレーター。ロード関数の上に付けると、その関数の中の書き込みを 1 つのトランザクションにまとめ、関数がエラーを返した時は、全部取り消します。	15 6. 不具合
`.endswith()`	文字列が、その文字で終わっていれば `True` を返すメソッド。	15 6. 不具合
`%`	Python の `7 % 2` は、割った余り（`1`）です。	15 6. 不具合
`.warning()`	`ロガー.warning(文)` は、ロガー（ログを書く窓口）に「注意」の重さでログを書くメソッド。`info` より重い知らせに使います。	15 7. 開発・運用の整理
`finally`	`try … except` のあと、成功しても失敗しても最後に必ず動く所。	15 8. テスト

コード	意味と使い方	初めて出てくる所
`.mkdtemp()`	`tempfile.mkdtemp()` は、OS の一時フォルダの中に、新しい空のフォルダを作って、その場所を返す関数。`prefix=` は名前の頭です。動かすたびに別のフォルダになるので、前のテストで作った物が混ざりません。	15 8. テスト
`.write_text()`	`Path` のファイルに、文字列を書くメソッド。`encoding="utf-8"` を付けます。	15 8. テスト
`.mkstemp()`	`tempfile.mkstemp()` は、一時フォルダに空のファイルを作り、（開いたファイルの番号, ファイルの場所）の 2 つを返す関数。番号は `os.close()` で閉じてから、場所の方を使います。	15 8. テスト
`.close()`	接続やファイルを閉じるメソッド。	15 8. テスト
`.TemporaryDirectory()`	`with tempfile.TemporaryDirectory() as 名前:` で、一時フォルダを作り、`with` の中だけ使えるようにする書き方。`with` を出ると、フォルダは中身ごと消えます。	15 8. テスト
`.write_bytes()`	`Path` のファイルに、バイトの並びをそのまま書くメソッド。	15 8. テスト
`dict()`	辞書を作る関数。`dict(BT, employee_no=3001, name='n')` は、辞書 `BT`（休憩時間の 24 項目）の中身に、`employee_no` などを足した、新しい辞書を作ります。	15 8. テスト
`sorted()`	並べかえた**新しい**リストを返す関数。`sorted(def_choice.objects.values_list('def_select', flat=True))` は、作業詳細の選択肢の文字を、並べかえたリストにします（テストで、並びの順番に関係なく中身をくらべるため）。	15 8. テスト
`b"…"`	`b'data'` は、文字ではなく「バイトの並び」（`bytes`）を作る書き方です。ファイルに書く中身や、ダウンロードで返ってきた中身（`b''.join(response.streaming_content)`）とくらべるのに使います。	15 8. テスト
`{キー: 値 for …}`	`{f'break_time{i}{s}': '#12001300' for i in range(1, 7) for s in ('', '_over1', '_over2', '_over3')}` は、「1 つずつから、キーと値の組を作って、辞書にする」書き方です。ここでは、休憩時間の 24 項目の辞書を、1 行で作っています。	15 8. テスト

Django

コード	意味と使い方	初めて出てくる所
`.urls`	`admin.site.urls` は、Django の管理画面の URL の一覧です。`path("admin/", admin.site.urls)` で、`/admin/` に管理画面を出します。	00 6. 実コード②：kosu/urls.py
`.as_view()`	クラスで作った画面（ビュー）を、URL の決まりに登録できる形にするメソッド。	00 6. 実コード②：kosu/urls.py
`path()`	URL と、それに答える係（ビュー）を結び付ける関数。`path('kosu_update/<int:pk>/', kosu_views.KosuUpdate.as_view(), name='kosu_update')` は「`kosu_update/5/` のような URL が来たら `KosuUpdate` に渡す。この決まりの名前は `kosu_update` 」という意味です。	00 6. 実コード②：kosu/urls.py
`<int:名前>`	Django の URL の `'kosu_update/<int:pk>/'` の `<int:pk>` は、「ここに来た数を、`pk` という名前で、ビューに渡す」書き方です。`int` は数だけ受け付けます（`str`・`slug`・`uuid` もあります）。	00 6. 実コード②：kosu/urls.py
`static()`	`urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)` は、開発中に、アップロードされたファイル（MEDIA）を URL で見られるようにする Django の道具です。本番では Web サーバーなどが代わりに配ります。	00 6. 実コード②：kosu/urls.py
`JsonResponse`	Django の `JsonResponse({"status": "ok"})` は、辞書を JSON にして返す答えです。	00 7. 実コード③：main_views.py の Login
`View`	Django の「ビュー（リクエストを受けて返事を作る物）」の土台のクラス。`class 名前(View):` と書いて、`def get(self, request):` のように、HTTP の種類ごとのメソッドを作ります。	00 7. 実コード③：main_views.py の Login
`.filter()`	Django の問い合わせで、条件に合う行だけに絞るメソッド。`kosu_division.objects.filter(kosu_name=def_ver)` は「工数区分定義の名前が `def_ver` の行だけ」。`項目名__条件`（`name__contains="山"` など）で、いろいろな条件を書けます。	00 7. 実コード③：main_views.py の Login

コード	意味と使い方	初めて出てくる所
`.exists()`	`member.objects.filter(employee_no=input_number).exists()` は、条件に合う行が 1 つでもあれば `True`、無ければ `False` を返すメソッドです。行を全部読み込まずに調べるので速いです（なお `os.path.exists(場所)` は「その場所にファイルやフォルダがあるか」で、別の物です）。	00 7. 実コード③：main_views.py の Login
`.order_by()`	問い合わせの結果を並べかえるメソッド。`order_by("id")` は id の小さい順、`order_by('-id')` は大きい順（`-` で逆＝新しい物から）。	00 7. 実コード③：main_views.py の Login
`.last()`	問い合わせの結果の最後の 1 件を取り出すメソッド。`kosu_division.objects.order_by("id").last()` は「id の順に並べた最後」＝いちばん新しい行です。1 件も無ければ `None` が返ります。	00 7. 実コード③：main_views.py の Login
`.body`	Django の `request.body` は、届いた要求の中身（バイトのまま）です。JSON なら `json.loads(request.body)` で読みます。	00 7. 実コード③：main_views.py の Login
`.objects`	`member.objects` は、その表の行を取り出す係（マネージャー）です。`.all()`（全部）・`.filter(…)`（しぼる）・`.get(…)`（1 つだけ）・`.create(…)`（作る）などを付けて使います。	00 7. 実コード③：main_views.py の Login
`.session`	`request.session` は、利用者ごとにサーバーが覚えておく入れ物（セッション）。`request.session['login_No'] = 番号` で「この人はこの番号でログインした」と覚え、次のリクエストでも読み出せます。	00 7. 実コード③：main_views.py の Login
`.all()`	`kosu_division.objects.all()` は、表の全部の行を取り出す（まだ読みに行かない「問い合わせ」を作る）メソッドです。	00 7. 実コード③：main_views.py の Login
`.delete()`	問い合わせに合う行（または 1 つの物）を、データベースから消すメソッド。消した件数が返ります。	00 7. 実コード③：main_views.py の Login
`.save()`	Django のモデル（1 行分の値）を、データベースに保存するメソッド。新しい物なら行を足し、もうある物なら行を変えます。	00 7. 実コード③：main_views.py の Login
`.DoesNotExist`	`member.DoesNotExist` は、`.get(…)` で、合う行が 1 つも無かった時のエラーです。`except member.DoesNotExist:` で受け止めます。	00 7. 実コード③：main_views.py の Login

コード	意味と使い方	初めて出てくる所
`.IntegerField()`	整数の列を表すモデルの項目。`models.IntegerField('従業員番号')` の最初の文字は、管理画面などに出る名前です。	00 8. 実コード④：models.py
`.CharField()`	`models.CharField('氏名', max_length=100)` は、短い文字列の列（最大の長さを必ず決める）を表すモデルの項目です。最初の `'氏名'` は、管理画面などに出る名前です。	00 8. 実コード④：models.py
`.BooleanField()`	はい・いいえ（`True`・`False`）の列を表すモデルの項目。	00 8. 実コード④：models.py
`.ForeignKey()`	ほかの表の 1 行を指す列（外部キー）を表すモデルの項目。`models.ForeignKey(member, verbose_name='氏名', null=True, on_delete=models.SET_NULL)` は「人員（`member`）の 1 人を指す。指していた人が消されたら、ここは空（NULL）にする」という意味です。	00 8. 実コード④：models.py
`.DateField()`	日付の列を表すモデルの項目。	00 8. 実コード④：models.py
`.TextField()`	長い文字列（何行もの文）の列を表すモデルの項目。`blank=True` を付けると、入力が空でもよくなります。	00 8. 実コード④：models.py
`.DateTimeField()`	日時の列を表すモデルの項目。`auto_now_add=True` で、作った時の日時を自動で入れます。	00 8. 実コード④：models.py
`.count()`	Django で、条件に合う行の数を数えるメソッド。`self.__class__.objects.count()` は、その表の全部の行の数。データベースに `SELECT count(*)` を送るので、全部を読み込むより速いです。	00 8. 実コード④：models.py
`include()`	URL の決まりの中で、ほかのファイル（アプリ）の URL の決まりを、まとめて取り込む関数。`path('api/', include('kosu.urls'))` は「`/api/` で始まる URL は、`kosu/urls.py` の決まりで続きを調べる」という意味です。	00 9. 実コード⑤：hozen_another/urls.py
`re_path()`	正規表現で URL の決まりを書く関数。`re_path(r'^(?!admin).*$', react_view)` は「`admin` で始まらない URL は全部、React の画面（`react_view`）を返す」。URL の決まりの最後に書きます。	00 9. 実コード⑤：hozen_another/urls.py
`.site`	`admin.site` は、Django の管理画面そのものです。`path('admin/', admin.site.urls)` で、`/admin/` に管理画面を出します。	00 9. 実コード⑤：hozen_another/urls.py

コード	意味と使い方	初めて出てくる所
`.path`	`request.path` は、届いたリクエストの URL の場所の部分（`/api/kosu_backup/` など。`?` から後ろは入りません）です。	00 12. 実コード⑦：settings.py の主要設定を1...
`.first()`	問い合わせの結果の最初の 1 件を返すメソッド。無ければ `None` です（`get()` と違い、エラーになりません）。	06 8. 三項演算子・f文字列
`.chunks()`	アップロードされたファイルを、少しずつ（かたまりごとに）読み出します。大きなファイルでも、メモリを一度に使い切りません。	06 11. with 文
`receiver()`	`@receiver(post_save, sender=member)` は「`member` が保存された直後（`post_save`）に、この関数を動かす」というシグナル（合図）の登録です。	06 13. デコレータ @
`@receiver`	関数の上に付けて、Django のシグナル（保存や削除の合図）を受け取る関数にします。`@receiver(合図, sender=モデル)`。	06 13. デコレータ @
`.create()`	Django で、新しい行を作ってデータベースに保存するメソッド。`AsyncTask.objects.create(task_id=task_id, status='pending')` は、作ると同時に保存します（`.save()` は要らない）。	06 16. 実コード全解説③：asynchronous_vie...
`.pk`	Django の `.pk` は、その行の主キー（ふつうは `id`）です。	06 18. 実コード全解説⑤：signals.py の差分計算
`Case`	問い合わせの中で「場合分け」をする道具。`Case(When(条件, then=値), …, default=値)` で、条件ごとに違う値を作り、並べ替えなどに使います。	07 5. 実コード全解説：member_views.py 全2...
`When`	`Case` の中の 1 つの場合。`When(shop='異動・退社', then=Value(1))` は「所属が『異動・退社』なら 1」。	07 5. 実コード全解説：member_views.py 全2...
`Value`	問い合わせの中で使う「決まった値」。`Value(1)` のように書きます。	07 5. 実コード全解説：member_views.py 全2...
`IntegerField`	`Case` などで作る値が「整数」だと Django に伝える物。`output_field=IntegerField()` と書きます。	07 5. 実コード全解説：member_views.py 全2...

コード	意味と使い方	初めて出てくる所
`.annotate()`	問い合わせの 1 行ごとに、計算した値を付け足すメソッド。`member.objects.annotate(shop_order=Case(…))` は、人員の 1 人ずつに「並べ替え用の番号（`shop_order`）」を付けます。	07 5. 実コード全解説：member_views.py 全2…
`列__条件=`	Django の ORM の `filter(work_day2__gte=start_day, work_day2__lte=end_day)` のように、列の名前の後ろに `__` と条件を付けると、いろいろな比べ方ができます。`__gte`（以上）・`__lte`（以下）・`__gt`（より大きい）・`__in`（並びのどれか）・`__icontains`（大文字・小文字を区別せず、含む）・`__isnull`（空か）など。	07 5. 実コード全解説：member_views.py 全2…
`__all__`	`fields = '__all__'` は「モデルの項目を全部使う」という指定です（シリアライザーやフォームで）。	07 6. 実コード全解説：serializers.py
`.get_response()`	ミドルウェア（リクエストの前後に必ず通る処理）で、次の係（最後はビュー）にリクエストを渡して返事を受け取る関数です。	07 10. 実コード全解説：middleware と apps…
`AppConfig`	アプリの設定のクラスの土台。`apps.py` で `class KosuConfig(AppConfig):` と書き、`def ready(self):` でシグナルを読み込むなど、起動の時の準備をします。	07 10. 実コード全解説：middleware と apps…
`FileResponse`	Django の `FileResponse(開いたファイル)` は、ファイルを、少しずつ送る答えです。添付ファイルのダウンロードに使います。	07 11. 実コード全解説：asynchronous_view…
`resolve()`	URL の道（`/api/members/`）から、どのビューにつながるかを調べます。`resolve(path).url_name` で、URL に付けた名前が分かります。	07 11. 実コード全解説：asynchronous_view…
`.FILES`	Django の `request.FILES` は、送られたファイルの入れ物です。`request.FILES.get("file")` で 1 つ取り出します。	07 11. 実コード全解説：asynchronous_view…
`.GET`	`request.GET` は、URL の `?` の後ろの値（`?task_id=abc`）の入れ物です。`request.GET.get('task_id')` で読みます。	07 11. 実コード全解説：asynchronous_view…
`.JSONField()`	JSON（入れ子になった辞書や並び）をそのまま入れる列を表すモデルの項目。このアプリでは、`jsonfield` という部品の `jsonfield.JSONField(blank=True, null=True)` を、操作履歴（`History`）の `changes`（何をどう変えたか）に使っています（Django にも同じ名前の `models.JSONField` があります）。	08 20. モデル⑧⑨⑩: AsyncTask / Opera…

コード	意味と使い方	初めて出てくる所
`IntegrityError`	データベースの決まり（同じ値は 2 つ入れない、など）を破った時のエラー。`except IntegrityError:` で受け止めます。	09 6. tasks.py 全解説①：工数バックアップ
`ObjectDoesNotExist`	`get()` で探した行が見つからない時のエラー。`except ObjectDoesNotExist:` で受け止めます。	09 6. tasks.py 全解説①：工数バックアップ
`get_wsgi_application()`	Django のアプリを、本番の Web サーバー（Gunicorn など）が呼べる形（WSGI）にして返します。`wsgi.py` の中で使います。	09 12. wsgi.py 全解説：Webサーバーとの玄関
`.assertEqual()`	テストで、2 つの値が同じかを確かめるメソッド。`self.assertEqual(res.status_code, 403)`。	12 9. 実コード全解説：TestModels
`.assertIn()`	テストで、値が入れ物の中にあるかを確かめるメソッド。`self.assertIn("title", res.json())`。	12 9. 実コード全解説：TestModels
`.status_code`	テストの答えの `res.status_code` は、HTTP の状態コード（`200`・`403` など）です。	12 10. APIの 401 / 403 / 200 を確かめ…
`@ensure_csrf_cookie`	その画面（ビュー）の返事に、CSRF の合言葉（`csrftoken` というクッキー）を必ず付ける印。React の画面は、このクッキーの値を、書き込みの時に `X-CSRFToken` という見出しに入れて送ります（`frontend/src/api/axios.ts`）。	15 5. 安全の問題
`method_decorator()`	関数用の印（デコレーター）を、クラスの画面（ビュー）のメソッドにも付けられるようにする物。	15 5. 安全の問題
`@method_decorator`	`@method_decorator(ensure_csrf_cookie, name="dispatch")` をクラスの上に付けて、そのクラスの画面に印を付けます。	15 5. 安全の問題
`.cycle_key()`	`request.session.cycle_key()` は、セッションの中身はそのままで、セッションの番号（クッキーの値）だけを新しくするメソッド。ログインの時に呼ぶと、ログインの前の番号を知っている人が、なりすませなくなります。	15 5. 安全の問題
`rotate_token()`	CSRF トークンを、新しい物に変える関数（`django.middleware.csrf` の中にある）。ログインの時に呼ぶと、ログイン前のトークンを知られていても、ログインした後は使えなくなります。Django 標準の `login()` の中でも、同じことをしています。	15 5. 安全の問題
`.user`	Django の `request.user` は、今ログインしている人（していなければ「だれでもない人」）です。`request.user.is_authenticated` で、ログインしているかが分かります。	15 5. 安全の問題

コード	意味と使い方	初めて出てくる所
`.update_or_create()`	条件に合う行があれば書きかえ、無ければ作るメソッド。`(物, 新しく作ったか)` の組が返ります。	15 5. 安全の問題
`.atomic()`	`with transaction.atomic():` の中のデータベースの変更を、「全部成功か、全部取り消し」にする書き方。	15 5. 安全の問題
`@transaction`	`@transaction.atomic` を関数の上に付けると、その関数の中のデータベースの変更が「全部成功か、全部取り消し」になります。	15 5. 安全の問題
`.set_rollback()`	`transaction.set_rollback(True)` は、今いる `transaction.atomic` の中の書き込みを、終わる時に全部取り消すよう印を付ける関数。`except` で例外を捕まえて普通に返すと、`atomic` は「うまくいった」とみなして確定してしまうので、取り消したい時はこれを呼びます。	15 5. 安全の問題
`.values()`	行を、モデルではなく、決めた列だけの辞書で受け取るメソッド。`member.objects.values('employee_no').annotate(n=Count('id'))` は、「従業員番号ごとに、何行あるか」を数える時の書き方です。	15 6. 不具合
`Count`	Django の `Count("id")` は、ORM の集計で、数を数えます。`.annotate(n=Count("id"))` で、グループごとの件数が付きます。	15 6. 不具合
`.UniqueConstraint()`	「この組み合わせの値は、表の中で重ならない」という決まりを表す物。モデルの `class Meta:` の `constraints` に書きます。`fields=['employee_no3', 'work_day2']` は「同じ人の、同じ就業日の行は、2 つ作れない」。`name=` は、決まりの名前（表の中で重ならない名前）です。	15 6. 不具合
`.AddConstraint()`	マイグレーションの中で、表に決まり（`UniqueConstraint` など）を足す操作。モデルの `class Meta:` の `constraints` を見て、`makemigrations` が書きます。	15 6. 不具合
`.savepoint()`	`transaction.savepoint()` は、トランザクションの途中に「ここまで戻れる印（セーブポイント）」を付けて、その印を返す関数。後で `transaction.savepoint_rollback(印)` を呼ぶと、印より後の書き込みだけを取り消せます。	15 6. 不具合
`.savepoint_rollback()`	`transaction.savepoint_rollback(sid)` は、`savepoint()` で付けた印 `sid` より後の書き込みだけを取り消す関数。ロードで「この 1 行だけ飛ばして、次の行へ進む」のに使います。	15 6. 不具合

コード	意味と使い方	初めて出てくる所
`.now()`	`timezone.now()` は、今の日時を、時差付きで返します（Django の設定に合わせた形）。Python の `datetime.now()` は時差が付かないので、Django ではこちらを使います。	15 6. 不具合
`.exclude()`	問い合わせから、条件に合う行を除くメソッド（`filter` の逆）。	15 6. 不具合
`.distinct()`	問い合わせの結果から、重なった値を 1 つにするメソッド。`.values_list('def_ver2', flat=True).distinct()` は、使われている定義の Ver 名を、重ならないように並べます。	15 6. 不具合
`.AlterField()`	マイグレーションの中で、表の列の形（文字数の上限・空を許すか、など）を変える操作。`models.py` の項目の書き方を変えると、`makemigrations` が書きます。	15 7. 開発・運用の整理
`.list()`	`env.list('ALLOWED_HOSTS', default=[...])` は、`django-environ` で、`a,b,c` のようにカンマで区切った環境変数を、並び（リスト）として読むメソッド。	15 7. 開発・運用の整理
`.refresh_from_db()`	手元にある 1 行分のデータ（モデルのインスタンス）の値を、データベースから読み直すメソッド。API でデータが書き換わったかを確かめる前に呼びます。	15 8. テスト
`.clear()`	`cache.clear()` は、Django のキャッシュ（一時的に覚えておく場所）を全部空にするメソッド。呼び出し回数の制限（`ScopedRateThrottle`）は回数をキャッシュに覚えるので、テストの前後で空にします。	15 8. テスト
`.values_list()`	問い合わせの結果を、タプル（値の組）の並びで返すメソッド。`flat=True` を付けると、1 つの列の値だけの並びになります。	15 8. テスト
`override_settings()`	`from django.test import override_settings` で読み込む、テストの間だけ設定を変える道具。`@override_settings(…)` の形で使います。	15 8. テスト
`@override_settings`	テストのメソッドの上に付けると、そのテストの間だけ、`settings.py` の値を変えられる印。`@override_settings(HELP_PATH='test-help')`。	15 8. テスト

コード	意味と使い方	初めて出てくる所
`Client`	Django のテスト用の、ブラウザの代わり（`django.test.Client`）。`Client(enforce_csrf_checks=True)` とすると、本物のブラウザと同じく、CSRF を確かめる状態で頼みを送れます（何も付けないと、確かめを切った状態）。	15 8. テスト
`.cookies`	テスト用のクライアントの `client.cookies` は、そのクライアントが覚えているクッキーです。`client.cookies['csrftoken'].value` で、配られた CSRF の合言葉の値が取れます。	15 8. テスト
`.json()`	テストで受け取った答え（レスポンス）の本文を、JSON として読んで、Python のリストや辞書にするメソッド。`self.assertEqual(res.json(), [])`。	15 8. テスト

Django REST framework

コード	意味と使い方	初めて出てくる所
`APIView`	Django REST framework の、API 用のビューの土台のクラス。`class LoginView(APIView):` と書いて、`def post(self, request):` などを作ると、JSON を受け取って JSON を返す API になります。	00 7. 実コード③：main_views.py の Login
`Response`	DRF の `Response(data)` は、API の答えです。辞書や並びを、JSON にして返します。	00 7. 実コード③：main_views.py の Login
`.data`	DRF の `serializer.data` は、モデルの値を、JSON にできる形（辞書）にした物です。`request.data` は、届いた中身（JSON やフォーム）です。	00 7. 実コード③：main_views.py の Login
`.query_params`	DRF の `request.query_params` は、URL の `?employee_no=123&shop=…` の部分の値です。`request.query_params.get('employee_no', None)` で、無い時は `None` を受け取ります。	06 7. 関数 def
`PageNumberPagination`	一覧をページに分けて返す仕組みの土台のクラス。このアプリは、これを元に `CustomPagination` を作っています（初めは `page_size = 20` ＝1 ページ 20 件。管理者設定の `menu_row` があれば、その件数）。	06 9. クラスとインスタンス

コード	意味と使い方	初めて出てくる所
`api_view()`	関数を、DRF の API の画面にする物。`@api_view(["GET", "POST"])` のように使います。	06 13. デコレータ @
`parser_classes()`	API が受け付ける送り方（JSON・ファイル付きのフォームなど）を決める物。	06 13. デコレータ @
`@api_view`	`@api_view(["GET", "POST"])` を関数の上に付けて、API にします。決めたメソッド以外は 405 で断られます。	06 13. デコレータ @
`@parser_classes`	`@parser_classes([MultiPartParser])` を付けて、ファイル付きのフォーム（multipart）を受け付けるようにします。	06 13. デコレータ @
`MultiPartParser`	DRF の `MultiPartParser` は、ファイルの付いたフォーム（`multipart/form-data`）を読む係です。	06 13. デコレータ @
`JSONParser`	JSON で送られてきたデータを読む係。`@parser_classes([…, JSONParser, …])` で、そのビューが受け取れるデータの形を決めます。	06 13. デコレータ @
`FormParser`	ふつうのフォーム（`名前=値&…` の形）で送られてきたデータを読む係。	06 13. デコレータ @
`.paginate_queryset()`	`paginator.paginate_queryset(一覧, request)` は、リクエストの `?page=2` などを見て、そのページの分だけを取り出します。	07 5. 実コード全解説：member_views.py 全2…
`.is_valid()`	送られてきたデータが、シリアライザー（入力の確かめ係）の決まりに合うかを調べるメソッド。`raise_exception=True` で、合わない時は 400 の答えにします。	07 5. 実コード全解説：member_views.py 全2…
`AllowAny`	「だれでも使える」という確かめ（いつも通す）。全部の API に「ログインが要る」をかけた後で、ログイン画面のように、ログインしていなくても使う API にだけ `permission_classes = [AllowAny]` と付けて外します。	15 5. 安全の問題
`ScopedRateThrottle`	同じ所（IP アドレス）からの呼び出しの回数を数えて、上限を超えたら 429（呼びすぎ）を返す部品。ビューに `throttle_classes = [ScopedRateThrottle]` と `throttle_scope = 'help'` を書くと、`settings.py` の `REST_FRAMEWORK` の `DEFAULT_THROTTLE_RATES` の `'help': '5/hour'`（1 時間に 5 回）が上限になります。	15 5. 安全の問題
`HasAuthority`	`kosu/permissions.py` にある、権限の確かめの部品。ログインしていて、人員の「権限（`authority`）」が ON の人だけを通します（足りない時は 403）。ビューに `permission_classes = [HasAuthority]` と書いて使います。	15 5. 安全の問題

コード	意味と使い方	初めて出てくる所
`.validated_data`	DRF の `serializer.validated_data` は、確かめ（ `is_valid()` ）を通った後の、安全な値です。	15 5. 安全の問題
`permission_classes()`	`from rest_framework.decorators import permission_classes` で読み込む、関数の API に確かめを付ける道具。`@permission_classes([…])` の形で使います。クラスの API では、`permission_classes = [...]` という変数に書きます。	15 5. 安全の問題
`@permission_classes`	`@api_view` で書いた関数の API に、「だれが使えるか」の確かめを付ける印。`@permission_classes([IsAdministrator])` で、管理者だけにします。`@api_view` のすぐ下に書きます。	15 5. 安全の問題

pandas・openpyxl

コード	意味と使い方	初めて出てくる所
`.Workbook()`	`openpyxl.Workbook()` は、新しい空の Excel のファイル（ブック）を作ります。`wb.active` が最初のシートです。	09 6. tasks.py 全解説①：工数バックアップ
`.DataFrame()`	`pd.DataFrame(データ, columns=[列の名前…])` は、pandas の表（データフレーム）を作ります。Excel のシートのように、行と列で値を持ちます。	09 7. tasks.py 全解説②：予測データ出力
`.isin()`	`df['列'].isin(['', '#'])` は、その列の値が並びの中のどれかなら `True` を返します。前に `~` を付けると「どれでもない」になります。	09 7. tasks.py 全解説②：予測データ出力
`.drop_duplicates()`	重なっている行を消します。`subset=[列…]` で「この列が同じなら重なり」とし、`keep='first'` で最初の 1 行を残します。	09 7. tasks.py 全解説②：予測データ出力
`.ExcelWriter()`	`pd.ExcelWriter(ファイルの場所, engine='openpyxl')` は、pandas の表を Excel のファイルに書く係を作ります。`with` で使うと、最後に自動で保存して閉じます。	09 7. tasks.py 全解説②：予測データ出力
`.to_excel()`	`df.to_excel(writer, index=False)` は、表を Excel に書きます。`index=False` で、左の行番号の列を書きません。	09 7. tasks.py 全解説②：予測データ出力
`.load_workbook()`	`openpyxl.load_workbook(ファイルの場所)` は、Excel のファイルを読み込みます。	09 8. tasks.py 全解説③：ロード

コード	意味と使い方	初めて出てくる所
`.cell()`	`ws.cell(行, 列)` は、シートの 1 つのマスを取り出します（どちらも 1 から数えます）。`.value` がマスの中身です。	09 8. tasks.py 全解説③：ロード

正規表現

コード	意味と使い方	初めて出てくる所
`正規表現`	正規表現は、「こういう形の文字」を表すための、小さな書き方の決まりです。`^\s*#` は「行の頭から、空白がいくつかあって、`#` が来る」という形を表します。`-match`（PowerShell）・`re.match`（Python）・`/…/`（JavaScript）・`Regex::new`（Rust）などで、文字がその形に合うかを調べたり、合う所を取り出したりします。記号ごとの意味は、この一覧の「正規表現」の表にまとめています。	00 9. 実コード⑤：hozen_another/urls.py
`(?=…)`	正規表現の `(?= )` は「この後ろに、こういう形が続く時だけ」、`(?! )` は「この後ろに、こういう形が続かない時だけ」を表します（先読み）。`^(?!admin).*$` は「`admin` で始まらない物、全部」。中の文字は、合わせた文字には入りません。	00 9. 実コード⑤：hozen_another/urls.py
`*`	正規表現の `*` は、「すぐ前の物が、0 回以上くり返す」を表します。`\s*` は「空白が 0 個以上」、`.*` は「何でも、何文字でも」。	00 9. 実コード⑤：hozen_another/urls.py
`.`	正規表現の `.` は、「どんな 1 文字でも」を表します。`.` という文字そのものを表したい時は、`\.` と書きます。	00 9. 実コード⑤：hozen_another/urls.py
`^`	正規表現の `^` は、「行（文字）の頭」を表します。`^#` は「`#` で始まる」。	00 9. 実コード⑤：hozen_another/urls.py
`$`	正規表現の `$` は、「行（文字）の終わり」を表します。`\d+$` は「数字で終わる」。`^…$` と両方書くと、「全体がこの形か」を調べられます。	00 9. 実コード⑤：hozen_another/urls.py
`正規表現`	正規表現は、「こういう形の文字」を表すための、小さな書き方の決まりです。`^\s*#` は「行の頭から、空白がいくつかあって、`#` が来る」という形を表します。`-match`（PowerShell）・`re.match`（Python）・`/…/`（JavaScript）・`Regex::new`（Rust）などで、文字がその形に合うかを調べたり、合う所を取り出したりします。記号ごとの意味は、この一覧の「正規表現」の表にまとめています。	05 8. 実コード全解説：KosuList.tsx 全215行

日時の書式

コード	意味と使い方	初めて出てくる所
`%Y`	日時の書式の `%Y` は、4 けたの年（`2026`）です。`strftime("%Y-%m-%d")` → `2026-11-05`。	06 16. 実コード全解説③：asynchronous_vie…
`%m`	日時の書式の `%m` は、2 けたの月（`01` 〜 `12`）です。	06 16. 実コード全解説③：asynchronous_vie…
`%d`	日時の書式の `%d` は、2 けたの日（`01` 〜 `31`）です。	06 16. 実コード全解説③：asynchronous_vie…
`%H`	日時の書式の `%H` は、24 時間の形の時（`00` 〜 `23`）です。	08 7.【最重要】休憩文字列 #HHMMHHMM の読み方
`%M`	日時の書式の `%M`（大文字）は、2 けたの分（`00` 〜 `59`）です。月の `%m`（小文字）と取り違えないようにします。	08 7.【最重要】休憩文字列 #HHMMHHMM の読み方
`%S`	日時の書式の `%S`（大文字）は、2 けたの秒（`00` 〜 `59`）です。	08 7.【最重要】休憩文字列 #HHMMHHMM の読み方
`%f`	日時の書式の `%f` は、秒より細かい部分（Python では 6 けたのマイクロ秒）です。同じ秒に作ったファイルの名前が、重ならないようにする時に使います。	08 7.【最重要】休憩文字列 #HHMMHHMM の読み方

Vitest・Testing Library

コード	意味と使い方	初めて出てくる所
`MemoryRouter`	React Router の部品で、URL をブラウザーではなくメモリーの中で持つ物。テストの中で `<MemoryRouter>` で囲むと、`Link` や `useNavigate` を使う部品をそのまま試せます。	12 4. 実コード全解説：Login.test.tsx 全 13…

コード	意味と使い方	初めて出てくる所
`.mock()`	`vi.mock("部品の場所", () => ({ … }))` は、その部品を、テスト用のにせ物（モック）に取り替えます。本物のサーバーや画像を使わずに試せます。	12 4. 実コード全解説：Login.test.tsx 全13…
`.fn()`	`vi.fn()` は、呼ばれたかどうか・何を渡されたかを覚える、にせの関数を作ります。	12 4. 実コード全解説：Login.test.tsx 全13…
`.importActual()`	`await vi.importActual("react-router-dom")` は、モックに取り替える前の本物の部品を読みます。「一部だけにせ物にして、残りは本物」にする時に使います。	12 4. 実コード全解説：Login.test.tsx 全13…
`.clearAllMocks()`	`vi.clearAllMocks()` は、にせの関数が覚えている「呼ばれた回数・渡された値」を全部消します。テストごとにまっさらにするため、`beforeEach` の中で呼びます。	12 4. 実コード全解説：Login.test.tsx 全13…
`describe()`	`describe("まとめの名前", () => { … })` は、関係のあるテストを 1 つのまとまりにします。	12 4. 実コード全解説：Login.test.tsx 全13…
`beforeEach()`	`beforeEach(() => { … })` は、同じまとまりの各テストの前に、毎回動かす準備を書きます。	12 4. 実コード全解説：Login.test.tsx 全13…
`.getByRole()`	役割（ボタン・見出し・リンクなど）と名前で、画面の部品を探すメソッド。`screen.getByRole("heading", { name: "ログイン" })` は「『ログイン』という見出し」。見つからないとテストは失敗します。	12 4. 実コード全解説：Login.test.tsx 全13…
`.toBeInTheDocument()`	`expect(要素).toBeInTheDocument()` は「その要素が画面にある」ことを確かめます（jest-dom の機能）。	12 4. 実コード全解説：Login.test.tsx 全13…
`.getByLabelText()`	`screen.getByLabelText("従業員番号")` は、その名前の見出し（ラベル）が付いた入力欄を探します。見つからないとテストは失敗します。	12 4. 実コード全解説：Login.test.tsx 全13…

コード	意味と使い方	初めて出てくる所
`<MemoryRouter>`	テストで、画面の部品を `<MemoryRouter>…</MemoryRouter>` で囲んで描きます。囲まないと、React Router の道具を使う部品はエラーになります。	12 4. 実コード全解説：Login.test.tsx 全13…
`it()`	`it("テストの名前", () => { … })` は、テストを 1 つ書きます。名前は日本語でもかまいません。	12 4. 実コード全解説：Login.test.tsx 全13…
`render()`	`render(<部品 />)` は、テストの中のにせのブラウザー（jsdom）に、部品を描きます。	12 4. 実コード全解説：Login.test.tsx 全13…
`expect()`	`expect(値)` は、「こうなっているはず」を確かめる書き方です。後ろに `.toBe(5)`・`.toBeInTheDocument()` などを付けます。違っていれば、その場でテストは失敗します。	12 4. 実コード全解説：Login.test.tsx 全13…
`.mockResolvedValueOnce()`	にせの関数が、次の 1 回だけ「成功して、この値を返す」ようにします。`mockedApi.post.mockResolvedValueOnce({ data: … })` で、サーバーが成功の返事をした場面を作れます。	12 4. 実コード全解説：Login.test.tsx 全13…
`.change()`	`fireEvent.change(入力欄, { target: { value: "12345" } })` は、入力欄に文字を入れた時の動きを起こします。	12 4. 実コード全解説：Login.test.tsx 全13…
`.click()`	`fireEvent.click(ボタン)` は、テストの中で、ボタンを押した時の動きを起こします。	12 4. 実コード全解説：Login.test.tsx 全13…
`.toHaveBeenCalledWith()`	`expect(にせの関数).toHaveBeenCalledWith("/")` は「その関数が、この値を渡されて呼ばれた」ことを確かめます。	12 4. 実コード全解説：Login.test.tsx 全13…
`waitFor()`	`await waitFor(() => { expect(…) })` は、中の確かめが通るまで、少しずつ待ちながらくり返します。通信の後に画面が変わるのを待つ時に使います。	12 4. 実コード全解説：Login.test.tsx 全13…

コード	意味と使い方	初めて出てくる所
`.findByRole()`	`await screen.findByRole("alert")` は、その役割（role）の要素が出てくるまで少し待って探します。通信の後に出るメッセージなどを探す時に使います。	12 4. 実コード全解説：Login.test.tsx 全13...
`.toHaveTextContent()`	`expect(要素).toHaveTextContent("文字")` は「その要素の中に、その文字がある」ことを確かめます。	12 4. 実コード全解説：Login.test.tsx 全13...
`.mockRejectedValueOnce()`	にせの関数が、次の 1 回だけ「失敗して、このエラーを出す」ようにします。サーバーがエラーを返した場面を作れます。	12 4. 実コード全解説：Login.test.tsx 全13...
`.toHaveBeenCalled()`	`expect(にせの関数).toHaveBeenCalled()` は「1 回以上呼ばれた」ことを確かめます。`.not.toHaveBeenCalled()` なら「1 回も呼ばれていない」。	12 15. 新しいテストの足し方
`.getByText()`	`screen.getByText("出勤")` は、画面に表示されている文字で、部品を探すメソッド。見つからないと、テストが失敗します。	15 8. テスト
`.getAllByRole()`	画面の中から、役割（`"option"` ＝ プルダウンの選択肢、など）が同じ物を全部探して、並び（配列）で返すメソッド。	15 8. テスト
`.toHaveLength()`	並び（配列）の個数が、その数であることを確かめるメソッド。`expect(並び).toHaveLength(10)` は「10 個ある」。	15 8. テスト
`.findByText()`	画面に、その文字が出てくるまで待って、見つかった物を返すメソッド（`await` で待つ）。読み込みが終わるのを待つ時に使います。	15 8. テスト
`.toBeChecked()`	チェックボックスに、チェックが入っていることを確かめるメソッド。`.not.toBeChecked()` は「入っていない」ことを確かめます。	15 8. テスト

Python のテスト

コード	意味と使い方	初めて出てくる所
`TestCase`	Django のテストの土台のクラス。`class 名前(TestCase):` の中に `def test_〜(self):` を書きます。テストごとに、試し用のまっさらなデータベースが使われます。	12 8. 実コード全解説：test_backend.py の土台

コード	意味と使い方	初めて出てくる所
`APIClient`	Django REST framework のテスト用の道具。`self.client = APIClient()` で作り、`self.client.post("/api/login/", データ, format="json")` のように、本物のサーバーを動かさずに API を呼べます。	12 8. 実コード全解説：test_backend.pyの土台
`MagicMock`	何を呼ばれても、それらしく返事をする、にせの物。`patch` と組み合わせて、メールの送信やファイルの保存などを、本当にはやらずに試します。	12 8. 実コード全解説：test_backend.pyの土台
`.assertLessEqual()`	`self.assertLessEqual(a, b)` は「a が b 以下」を確かめます。違えばテストは失敗します。	12 9. 実コード全解説：TestModels
`.assertFalse()`	`self.assertFalse(値)` は「値が偽（False）」を確かめます。	12 9. 実コード全解説：TestModels
`.assertTrue()`	`self.assertTrue(値)` は「値が真（True）」を確かめます。	12 10. APIの 401 / 403 / 200 を確かめ…
`.assertNotEqual()`	2 つの値が「違う」ことを確かめるメソッド。同じだと、テストが失敗します。	15 8. テスト
`.object()`	`mock.patch.object(物, '名前', 値)` を `with` で使うと、`with` の中だけ、その物の属性を別の値に差し替えます（`unittest.mock`）。テストで `ScopedRateThrottle.THROTTLE_RATES` を差し替えて、上限を 5 回にしています。	15 8. テスト
`.dict()`	`mock.patch.dict(os.environ, {'HELP_PATH': ''})` を `with` で使うと、`with` の中だけ、辞書（ここでは環境変数）の中身を書き換えます。`with` を出ると元に戻ります。	15 8. テスト

GitHub Actions

コード	意味と使い方	初めて出てくる所
`name:`	`name:` は、名前です。いちばん上の `name: Deploy Django + React to Azure` は、ワークフロー（自動の手順）の名前、`- name: Build React` は、手順 1 つの名前で、GitHub の Actions の画面に表示されます。	15 7. 開発・運用の整理

コード	意味と使い方	初めて出てくる所
`on:`	`on:` は、「いつ動かすか」です。`push:`（送った時）・`pull_request:`（プルリクエストの時）・`schedule:`（決まった時刻）などを並べます。	15 7. 開発・運用の整理
`push:`	`push:` は、「リポジトリに送った（push）時に動かす」指定です。	15 7. 開発・運用の整理
`branches:`	`branches:` の下に、ブランチの名前を並べて、「そのブランチに送った時だけ」としぼります。このアプリは `- master` なので、`master` に push した時だけ動きます。	15 7. 開発・運用の整理
`workflow_dispatch:`	`on:` の下の `workflow_dispatch:` は、「GitHub の Actions の画面の『Run workflow』ボタンで、手で動かせるようにする」指定です。	15 7. 開発・運用の整理
`jobs:`	`jobs:` の下に、動かす仕事（ジョブ）を並べます。ジョブは、ふつう同時に動きます。	15 7. 開発・運用の整理
`build-and-deploy:`	`jobs:` の下の `build-and-deploy:` は、仕事（ジョブ）の名前です。名前は自分で決めます。このアプリは、build・テスト・本番に出す、を 1 つの仕事でやっています。	15 7. 開発・運用の整理
`runs-on:`	`runs-on: ubuntu-latest` は、その仕事を、どの OS の機械で動かすかです。GitHub が用意した Linux（Ubuntu）の機械を、その時だけ借ります。	15 7. 開発・運用の整理
`env:`	`env:` の下に `名前: 値` を並べると、その手順（または仕事全体）の中だけで使える環境変数になります。手順の下に書けば、その手順の中だけで見えます。	15 7. 開発・運用の整理
`SECRET_KEY:`	`env:` の下の `SECRET_KEY: …` は、`SECRET_KEY` という環境変数を作る行です。Django の `settings.py` は、起動の時にこの値を読みます（無いと動きません）。	15 7. 開発・運用の整理
`${{ … }}`	`${{ }}` の中に書いた物は、GitHub Actions が、動かす時に値に置き換えます。`${{ secrets.SECRET_KEY }}` は、リポジトリの Secrets に入れた `SECRET_KEY` の値、`${{ github.event_name }}` は、何をきっかけに動いたか（`push`・`pull_request` など）です。	15 7. 開発・運用の整理
`secrets.名前`	`secrets.SECRET_KEY` のように書くと、リポジトリの Settings → Secrets and variables → Actions に入れた、秘密の値を読めます。値は画面やログには出ません（`***` になります）。	15 7. 開発・運用の整理
`DEBUG:`	`env:` の下の `DEBUG: "False"` は、`DEBUG` という環境変数を作る行です。`"False"` と引用符で囲むのは、YAML が `False` を「文字」ではなく「真偽の値」と読まないようにするためです。	15 7. 開発・運用の整理

コード	意味と使い方	初めて出てくる所
`DB_NAME:`	`env:` の下の `DB_NAME: ${{ secrets.DB_NAME }}` は、`DB_NAME` という環境変数に、GitHub に登録した秘密の値（`secrets`）を入れる行です。`DB_USER`・`DB_PASSWORD`・`DB_HOST` も同じ形です。	15 7. 開発・運用の整理
`DB_USER:`	`env:` の下の `DB_USER: ${{ secrets.DB_USER }}` は、データベースにつなぐ人の名前を、GitHub に登録した秘密の値から入れる行です。	15 7. 開発・運用の整理
`DB_PASSWORD:`	`env:` の下の `DB_PASSWORD: ${{ secrets.DB_PASSWORD }}` は、データベースのパスワードを、GitHub に登録した秘密の値から入れる行です。	15 7. 開発・運用の整理
`DB_HOST:`	`env:` の下の `DB_HOST: ${{ secrets.DB_HOST }}` は、データベースのサーバーの住所を、GitHub に登録した秘密の値から入れる行です。	15 7. 開発・運用の整理
`HELP_PATH:`	`env:` の下の `HELP_PATH: …` は、ヘルプ画面のパスワードを入れる環境変数を作る行です。テストの手順では、テストのためだけの値を入れます。	15 7. 開発・運用の整理
`steps:`	`steps:` の下に、その仕事の手順を、上から順に `-` で並べます。	15 7. 開発・運用の整理
`-`	YAML の行の頭の `-` は、「並びの 1 つ」を表します。`steps:` の下の `- name: …` は、手順の 1 つ目・2 つ目…です。字下げ（空白の数）で、どこに入るかが決まるので、そろえて書きます。	15 7. 開発・運用の整理
`uses:`	`uses: actions/checkout@v4` は、ほかの人が作った決まった手順（アクション）を使う指定です。`actions/checkout` は、リポジトリのファイルを取ってくる手順、`@v4` はその版です。`actions/setup-python` は Python を、`actions/setup-node` は Node.js を入れる手順です。	15 7. 開発・運用の整理
`with:`	`with:` は、`uses:` で使うアクションに渡す設定です。	15 7. 開発・運用の整理
`python-version:`	`actions/setup-python` に渡す設定で、入れる Python の版です。`"3.12"` と引用符で囲むのは、`3.10` が数の `3.1` と読まれるような間違いを防ぐためです。	15 7. 開発・運用の整理
`node-version:`	`actions/setup-node` に渡す設定で、入れる Node.js の版です。`"22"` なら、22 系の新しい版が入ります。	15 7. 開発・運用の整理

コード	意味と使い方	初めて出てくる所
`run:`	`run:` は、その手順で動かすコマンドです。`run: |` と書くと、次の行から何行も書けます（`cd frontend` の次の行に `npm ci`、のように）。	15 7. 開発・運用の整理
`|`	`run: |` の `|` は、「次の行から、字下げした所までを、改行も含めてそのまま 1 つの文字にする」という印です。コマンドを何行も書く時に使います。	15 7. 開発・運用の整理
`app-name:`	`azure/webapps-deploy` に渡す設定で、送り先の Azure App Service の名前です（このアプリは `hozen-kosu-React`）。	15 7. 開発・運用の整理
`publish-profile:`	`azure/webapps-deploy` に渡す設定で、Azure に送るための「通行証」（発行プロファイル）です。値は秘密なので、Secrets から読みます。	15 7. 開発・運用の整理
`package:`	`azure/webapps-deploy` に渡す設定で、送るファイルです。このアプリは、全部を 1 つにまとめた `release.zip` を送ります。	15 7. 開発・運用の整理
`pull_request:`	`pull_request:` は、「プルリクエスト（変更を取り込んでほしいというお願い）が作られた・更新された時に動かす」指定です。	15 7. 開発・運用の整理
`#`	YAML（GitHub Actions の設定などの書き方）では、`#` から行の終わりまでがコメントです。	15 7. 開発・運用の整理
`test:`	`jobs:` の下の `test:` は、仕事（ジョブ）に付けた名前です（自分で決めます）。	15 7. 開発・運用の整理
`needs:`	`needs: test` は、「`test` の仕事が成功してから動かす」という順番の指定です。	15 7. 開発・運用の整理
`if:`	`if:` に条件を書くと、その条件が合う時だけ、その手順を動かします。`if: github.event_name != 'pull_request'` は「Pull Request がきっかけでない時だけ」（＝ master に push した時だけ本番に出す）。	15 7. 開発・運用の整理